

# CS 305: Computer Networks

## Fall 2025

### **Lecture 7: Transport Layer**

**Ming Tang**

Department of Computer Science and Engineering  
Southern University of Science and Technology (SUSTech)

# Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

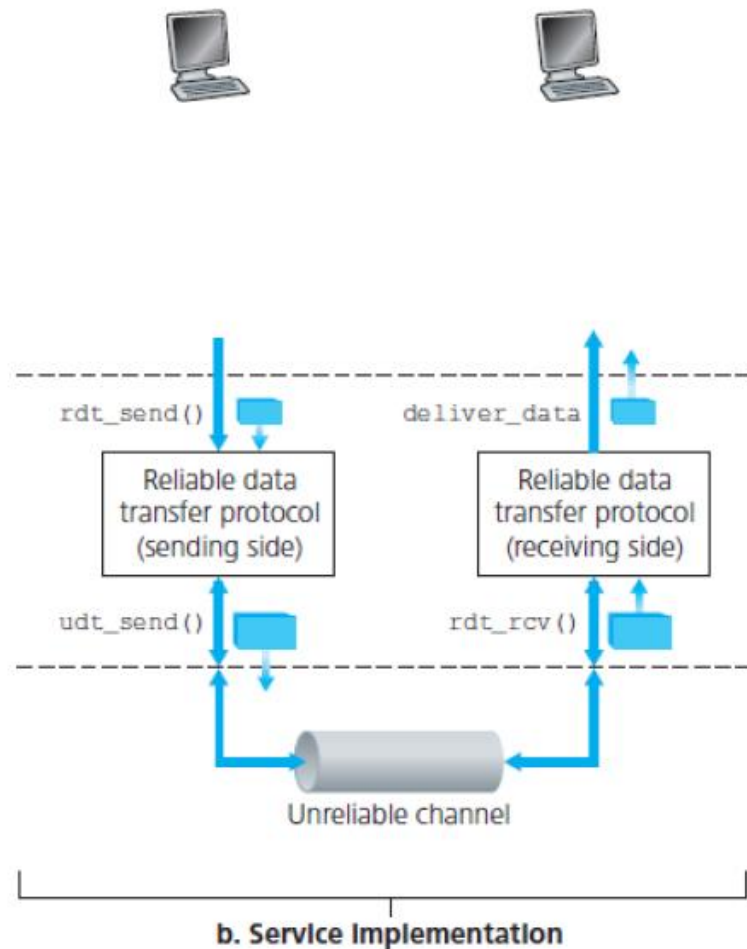
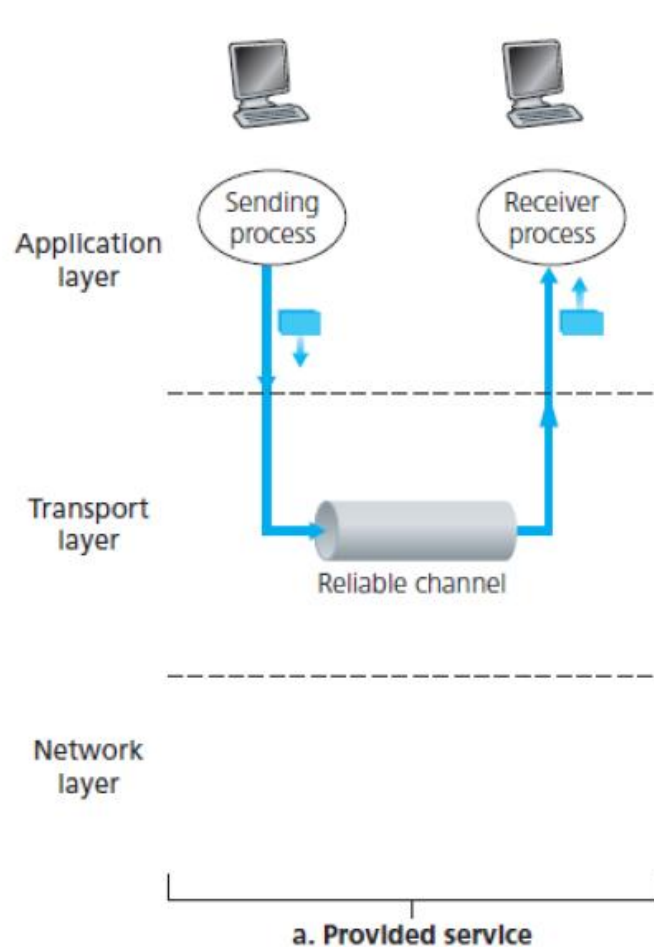
3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

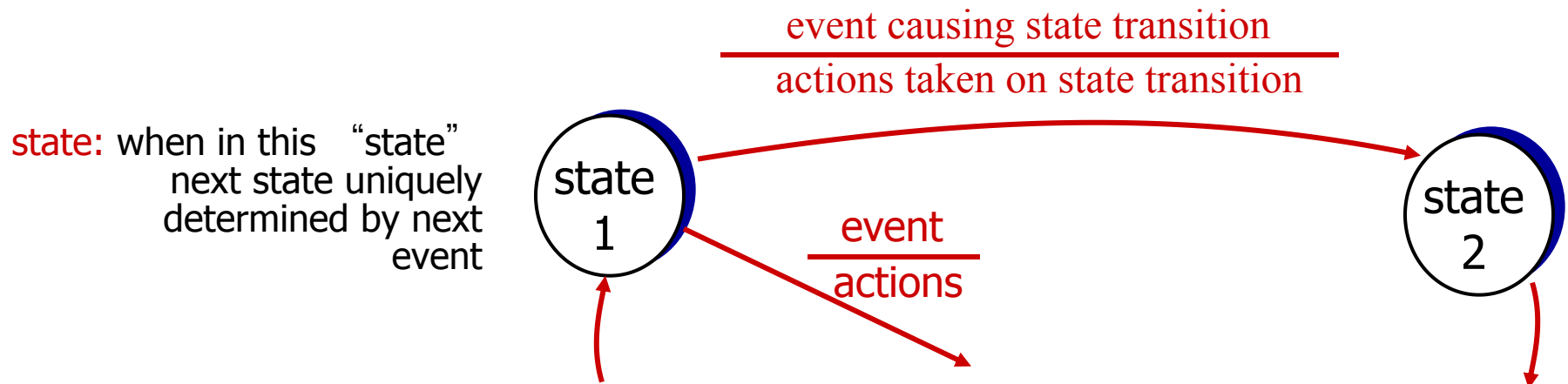
# Reliable Data Transfer (rdt)



# Reliable data transfer: getting started

We'll:

- ❖ **Incrementally** develop sender, receiver sides of reliable data transfer protocol (rdt)
- ❖ Consider only **unidirectional data transfer**
  - but control info will flow on both directions!
- ❖ Use finite state machines (FSM) to specify sender, receiver



# Overview

---

## Roadmap:

- ❖ Perfectly reliable channel: rdt1.0
- ❖ Channel with bit error:
  - bit error in packet: rdt 2.0
  - bit error in ACK: 2.1
  - NAK-free: 2.2
- ❖ Lossy channel: rdt 3.0

## Summary of Techniques

- Checksum
- Sequence number
- ACK packets
- Retransmission
- Timeout

# rdt1.0: reliable transfer over a reliable channel

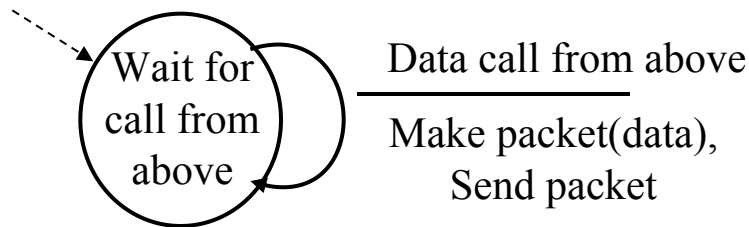
## ❖ Underlying channel **perfectly reliable**

- no bit errors
- no loss of packets

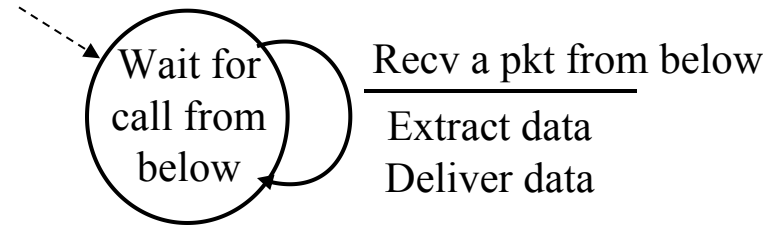


## ❖ **Rdt 1.0:**

- sender sends data into underlying channel
- receiver reads data from underlying channel
- Reliable channel, no need for feedback (no control message)

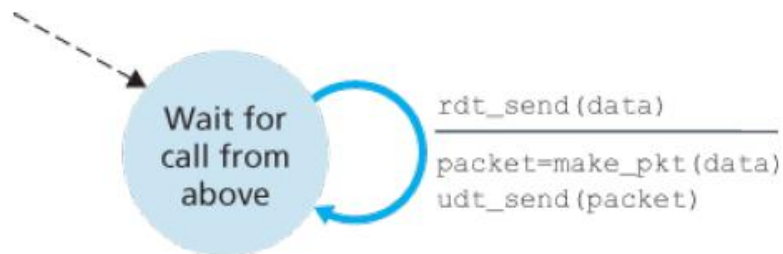
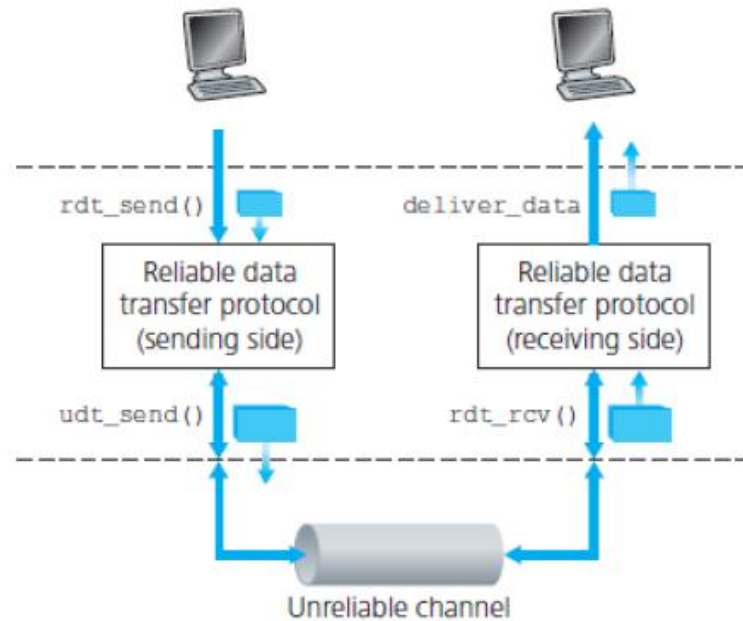


sender

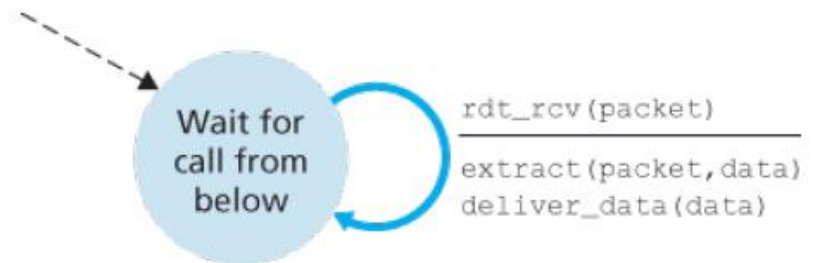


receiver

# rdt1.0: reliable transfer over a reliable channel



sender

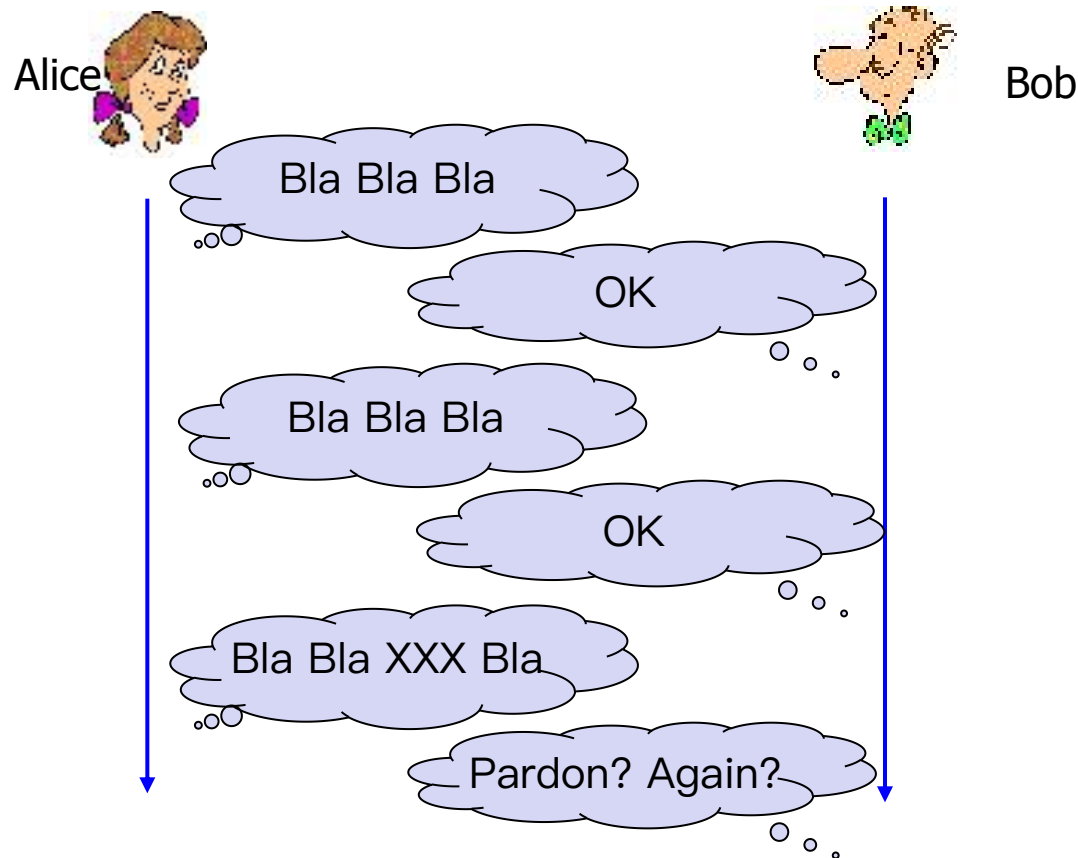


receiver

# rdt2.0: channel with bit errors

- ❖ Underlying channel may **flip bits** ( $0 \rightarrow 1$ ) in packet

How do humans recover from “errors” during conversation?





# rdt2.0: channel with bit errors

- ❖ Underlying channel may **flip bits** ( $0 \rightarrow 1$ ) in packet

How do humans recover from “errors” during conversation?

- ❖ **The question:** how to recover from errors?
  - *acknowledgements (ACKs)*: receiver explicitly tells sender that pkt received OK
  - *negative acknowledgements (NAKs)*: receiver explicitly tells sender that pkt had errors
  - sender retransmits pkt on receipt of NAK
- ❖ new mechanisms in **rdt2.0** (beyond **rdt1.0**):
  - error detection
  - receiver feedback: control msgs (ACK,NAK) rcvr->sender
  - retransmission

# rdt2.0: channel with bit errors

## ❖ Key mechanisms:

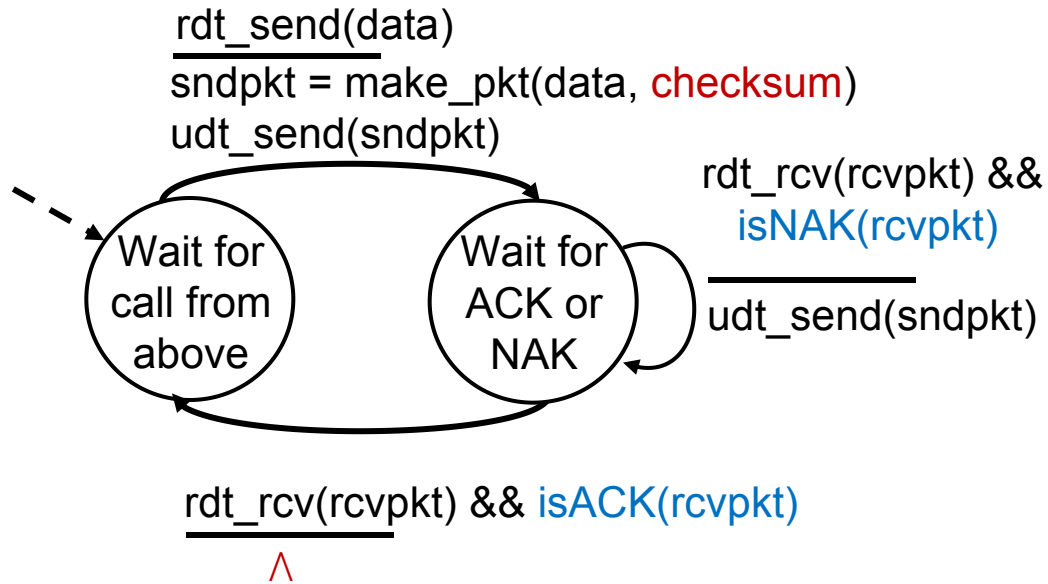
- **error detection**
- **feedback**: control msgs (ACK, NAK) from receiver to sender
- retransmission

## ❖ Error detection: checksum

## ❖ Feedback messages:

- *acknowledgements (ACKs)*: receiver explicitly tells sender that pkt received OK
- *negative acknowledgements (NAKs)*: receiver explicitly tells sender that pkt had errors
- sender retransmits pkt on receipt of NAK

# rdt2.0: FSM specification

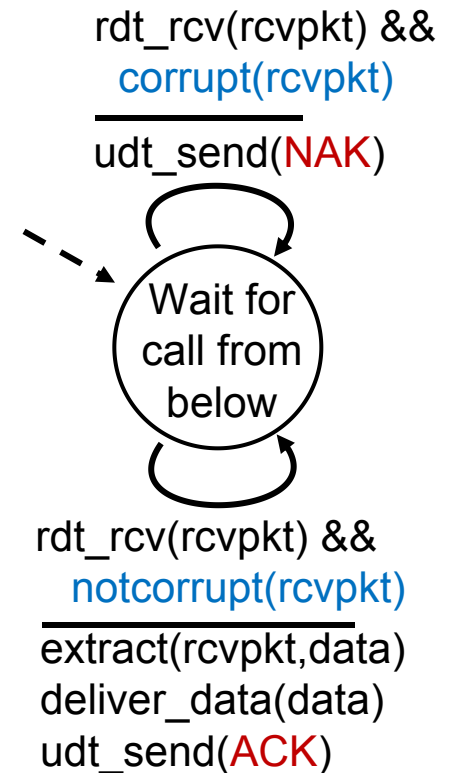


sender

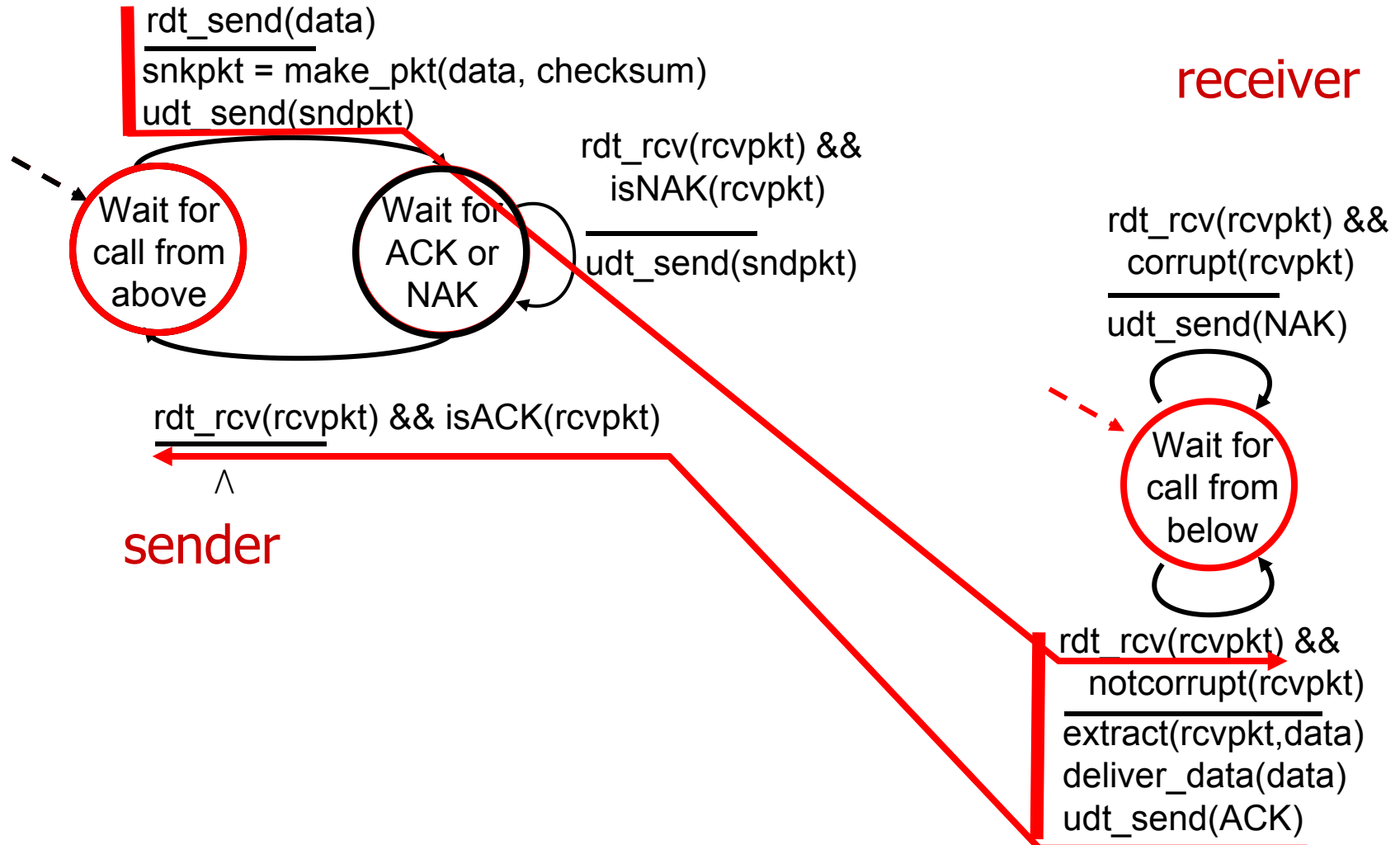
## Stop and wait

Sender sends one packet,  
then waits for receiver  
response

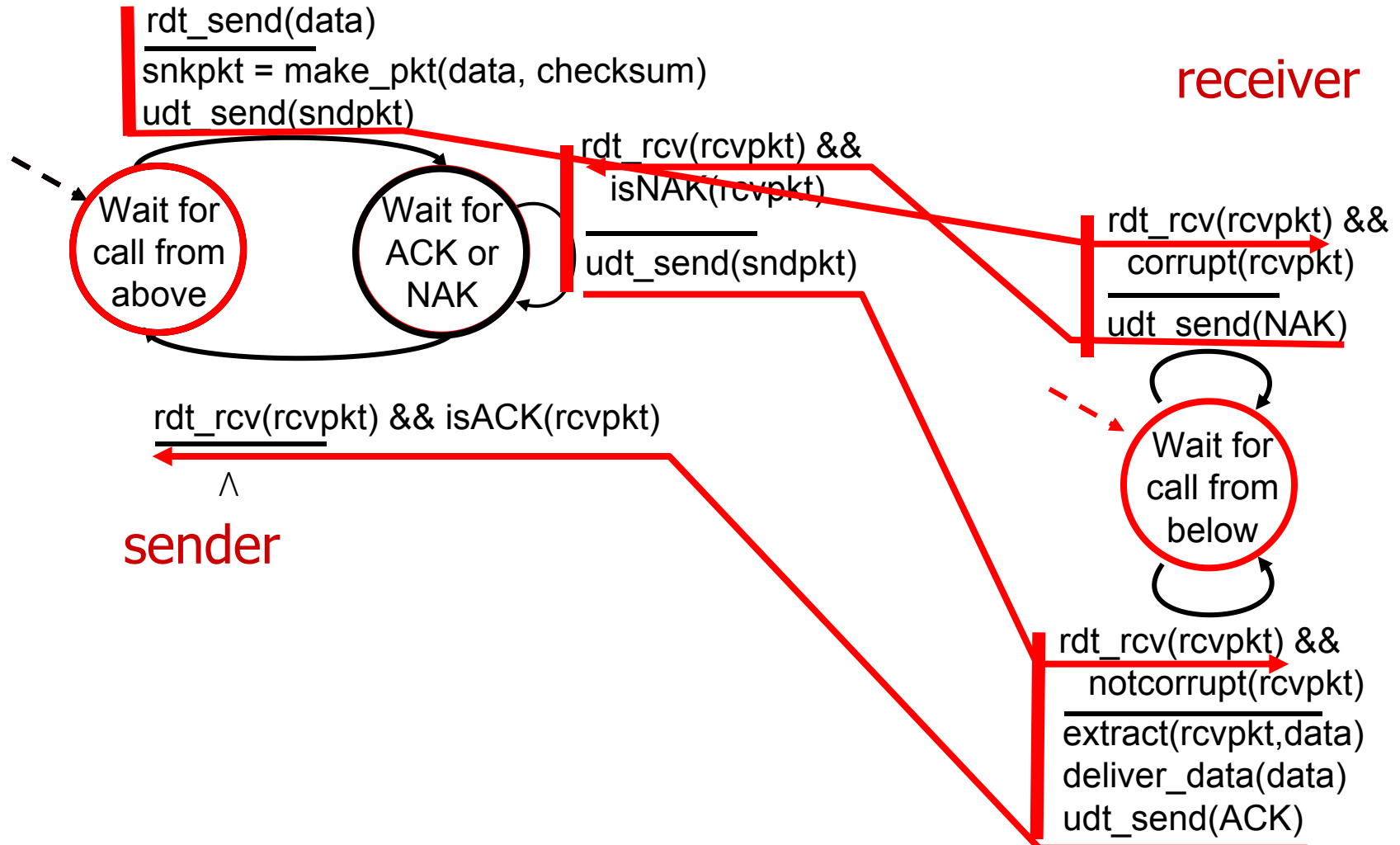
receiver



# rdt2.0: operation with no errors



# rdt2.0: error scenario



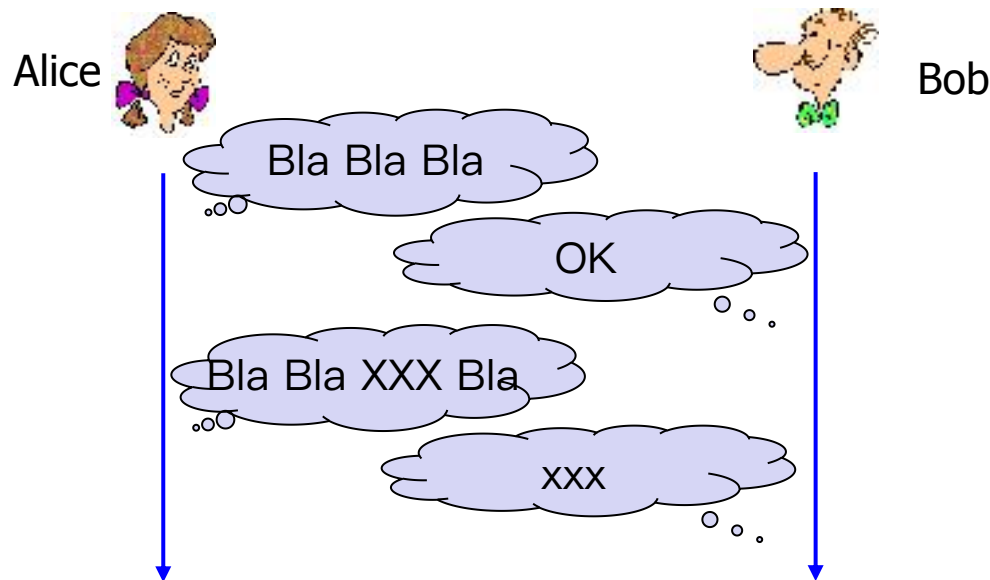
# rdt2.0 has a fatal flaw!

The possibility that ACK or NAK packet could be corrupted:

- ❖ Checksum bits

Handling corrupted ACKs or NAKs:

- ❖ Option 1: “blabla...”, “OK”, “What did you say?”, “OK”
  - “What did you say?”, “What did you say?”, ...
- ❖ Option 2: add enough checksum to recover
- ❖ Option 3: when garbled ACK or NAK, retransmit



# rdt2.0 has a fatal flaw!

The possibility that ACK or NAK packet could be corrupted:

- ❖ Checksum bits

Handling corrupted ACKs or NAKs:

- ❖ Option 1: “blabla...”, “OK”, “What did you say?”, “OK”
  - “What did you say?”, “What did you say?”, ...
- ❖ Option 2: add enough checksum to recover
- ❖ **Option 3: when garbled ACK or NAK, retransmit**

**Problem:** can't just retransmit: new data or retransmission?  
possible duplicate

**Handling duplicates:**

- ❖ sender retransmits current pkt if ACK/NAK corrupted
- ❖ sender adds *sequence number* to each pkt
- ❖ receiver discards (doesn't deliver up) duplicate pkt

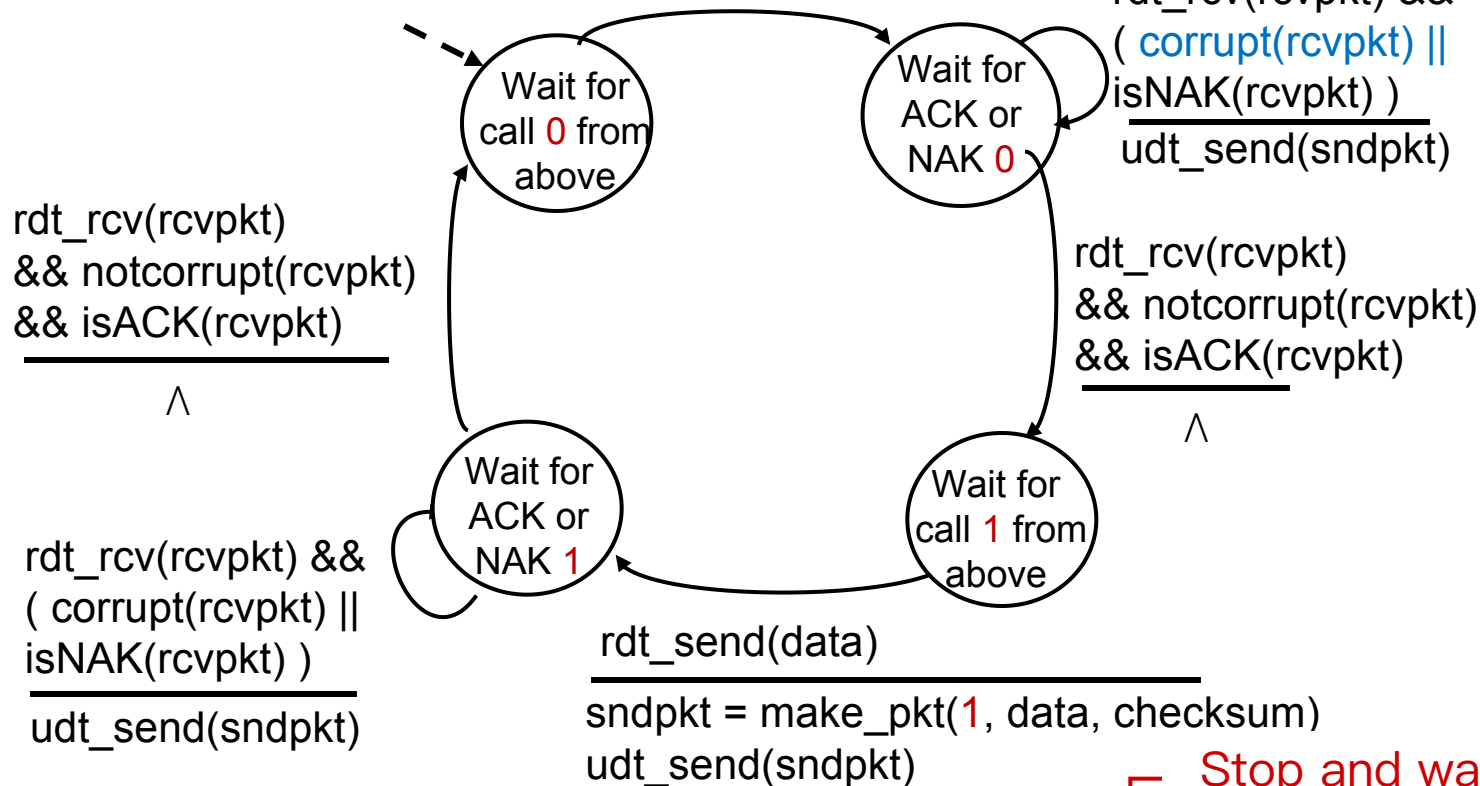
# rdt2.1: sender, handles garbled ACK/NAKs

sender

rdt\_send(data)

sndpkt = make\_pkt(0, data, checksum)

udt\_send(sndpkt)



Stop and wait

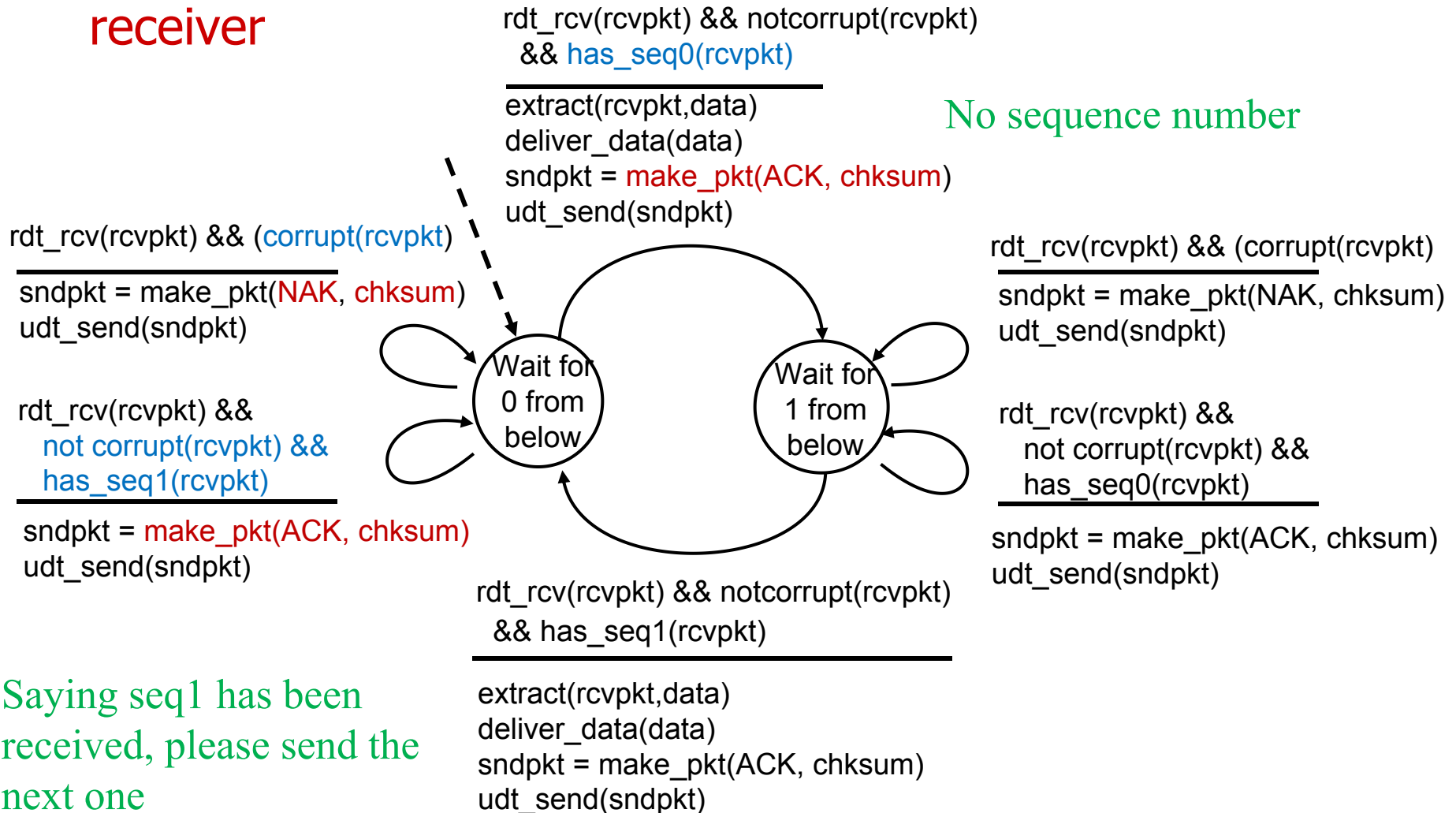
Sender sends one packet, then waits for receiver response

Two sequence number would be sufficient!



# rdt2.1: receiver, handles garbled ACK/NAKs

## receiver



# rdt2.1: discussion

## sender:

- ❖ seq # added to pkt
- ❖ must check if received ACK/NAK corrupted
- ❖ twice as many states
  - state must “remember” whether “expected” pkt should have seq # of 0 or 1

## receiver:

- ❖ must check if received packet is duplicate
  - state indicates whether 0 or 1 is expected pkt seq #
- ❖ note: receiver can *not* know if its last ACK/NAK received OK at sender

Two seq. #'s (0,1) will suffice. Why?

# rdt2.2: a NAK-free protocol

- ❖ same functionality as rdt2.1, **using ACKs only**
- ❖ instead of NAK, receiver sends ACK for last pkt received OK
  - receiver must *explicitly* include seq # of pkt being ACKed
- ❖ **duplicate ACK** at sender results in same action as NAK: *retransmit current pkt*

# rdt2.2: sender, receiver fragments

sender

rdt\_send(data)

sndpkt = make\_pkt(0, data, checksum)

udt\_send(sndpkt)

Wait for  
call 0 from  
above

Wait for  
ACK  
0

sender FSM  
fragment

rdt\_rcv(rcvpkt) &&

( corrupt(rcvpkt) ||  
isACK(rcvpkt,1) )

udt\_send(sndpkt)

rdt\_rcv(rcvpkt)  
&& notcorrupt(rcvpkt)  
&& isACK(rcvpkt,0)

∧

rdt\_rcv(rcvpkt) &&  
( corrupt(rcvpkt) ||  
has\_seq1(rcvpkt))

udt\_send(sndpkt)

Wait for  
0 from  
below

receiver FSM  
fragment

rdt\_rcv(rcvpkt) && notcorrupt(rcvpkt)  
&& has\_seq1(rcvpkt)

extract(rcvpkt,data)

deliver\_data(data)

sndpkt = make\_pkt(ACK, 1, chksum)

udt\_send(sndpkt)

receiver

# rdt3.0: channels with errors *and* loss

New assumption: underlying channel can also lose packets (data, ACKs)

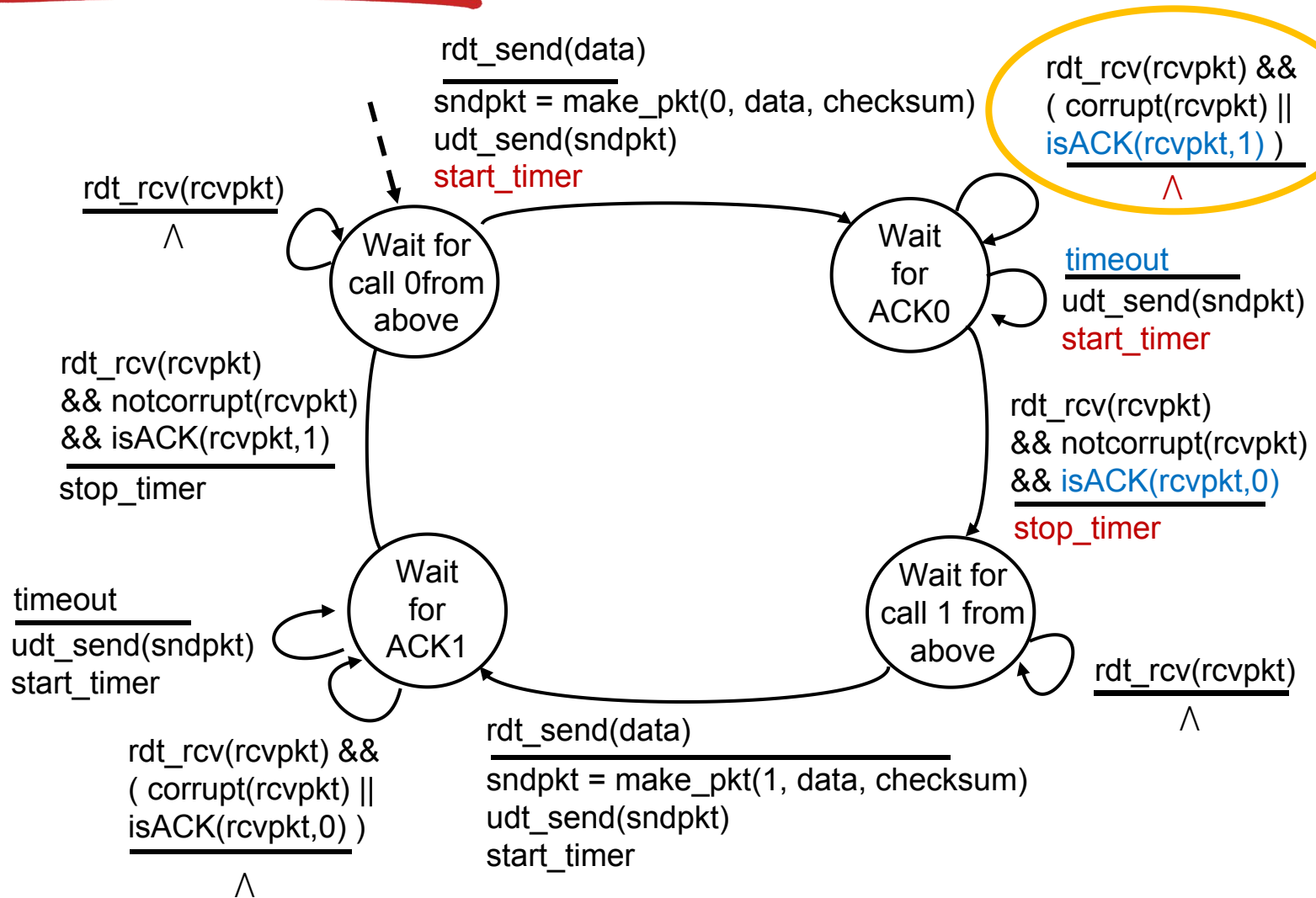
- checksum, seq. #, ACKs, retransmissions will be of help ... but not enough

Approach: sender waits “reasonable” amount of time for ACK

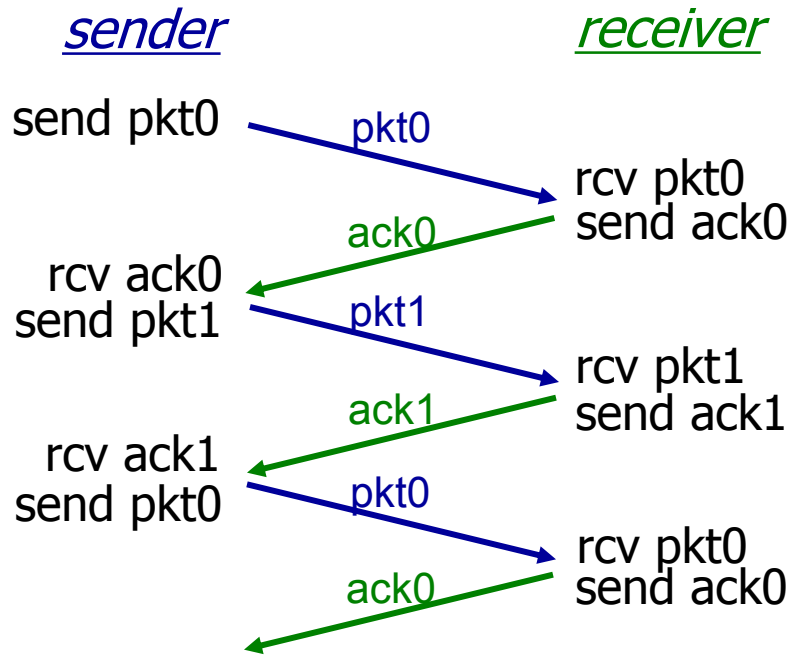
- ❖ retransmits if no ACK received in this time
- ❖ if pkt (or ACK) just delayed (not lost):
  - retransmission will be duplicate, but seq. #'s already handles this
  - receiver must specify seq # of pkt being ACKed
- ❖ requires countdown timer
  - start timer, timer interrupt, stop timer

How long should the sender wait?

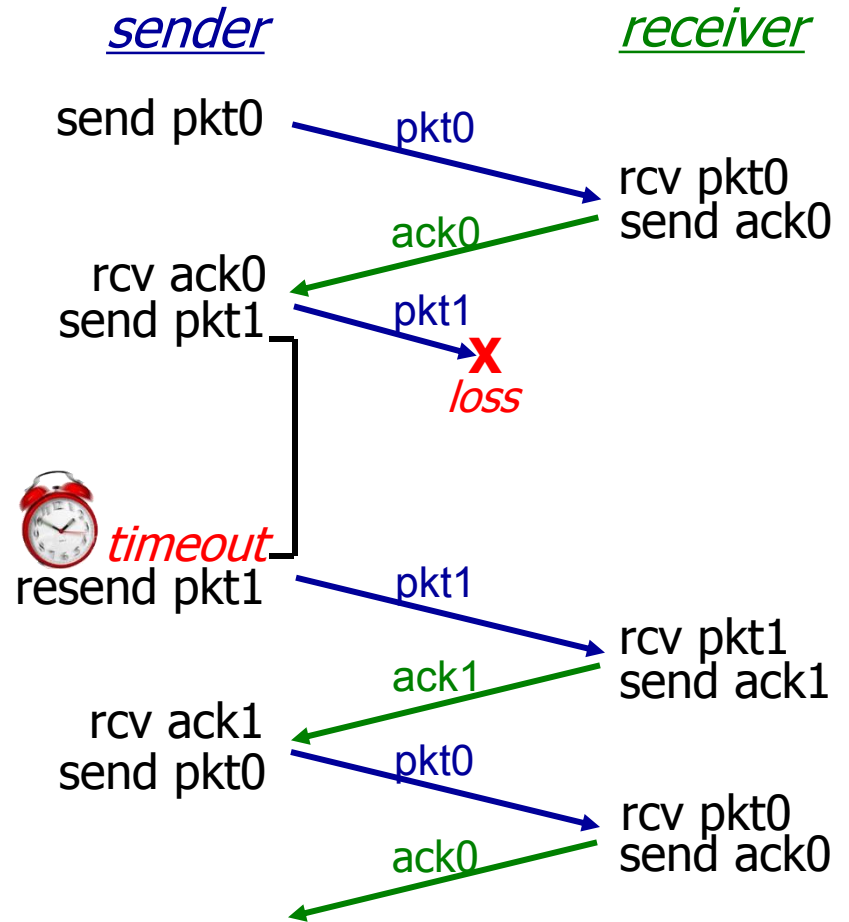
# rdt3.0 sender



# rdt3.0 in action

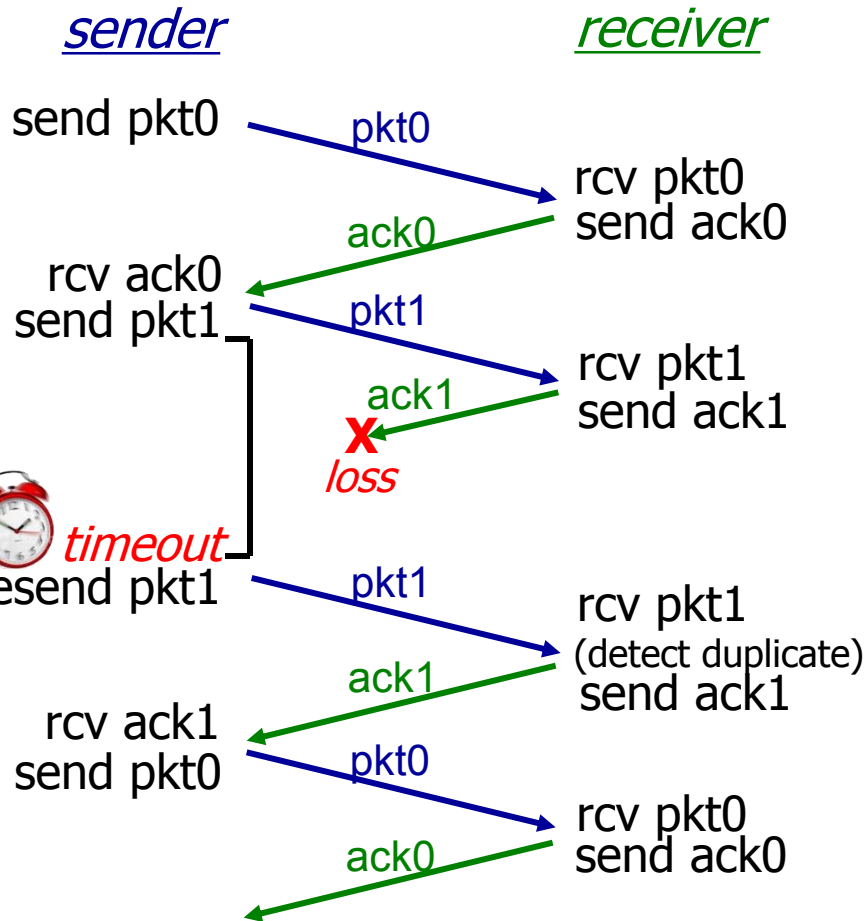


(a) no loss

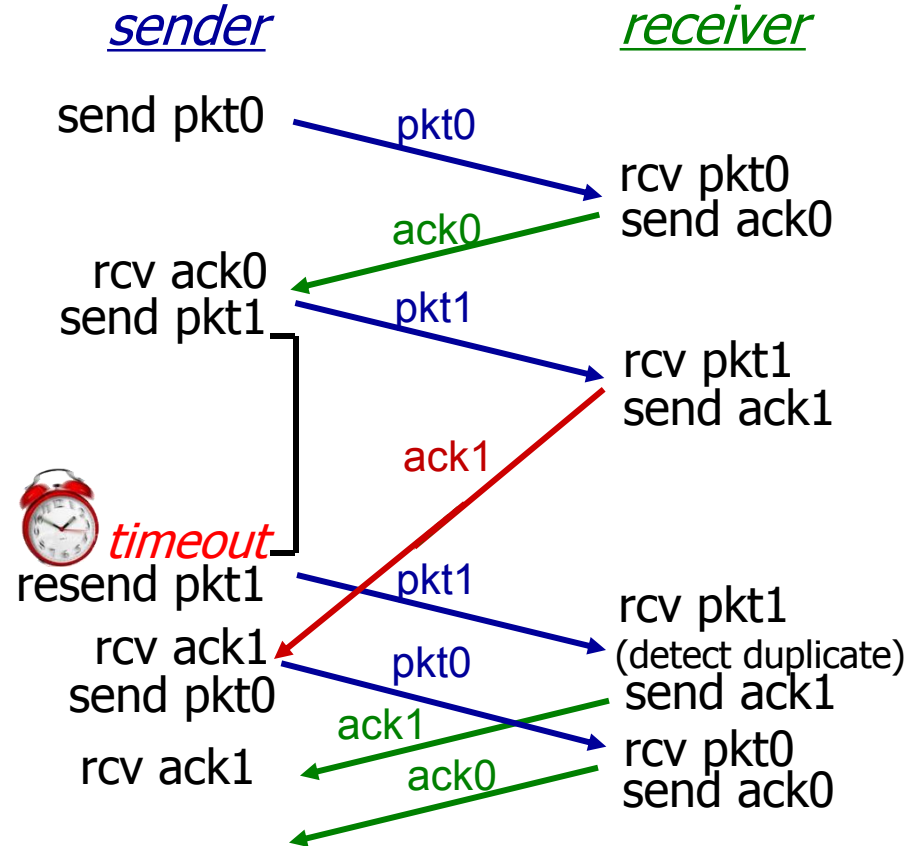


(b) packet loss

# rdt3.0 in action



(c) ACK loss



(d) premature timeout/ delayed ACK



# Summary

## Roadmap:

- ❖ Perfectly reliable channel: rdt1.0
- ❖ Channel with bit error:
  - bit error in packet: rdt 2.0
  - bit error in ACK: 2.1
  - NAK-free: 2.2
- ❖ Lossy channel: rdt 3.0

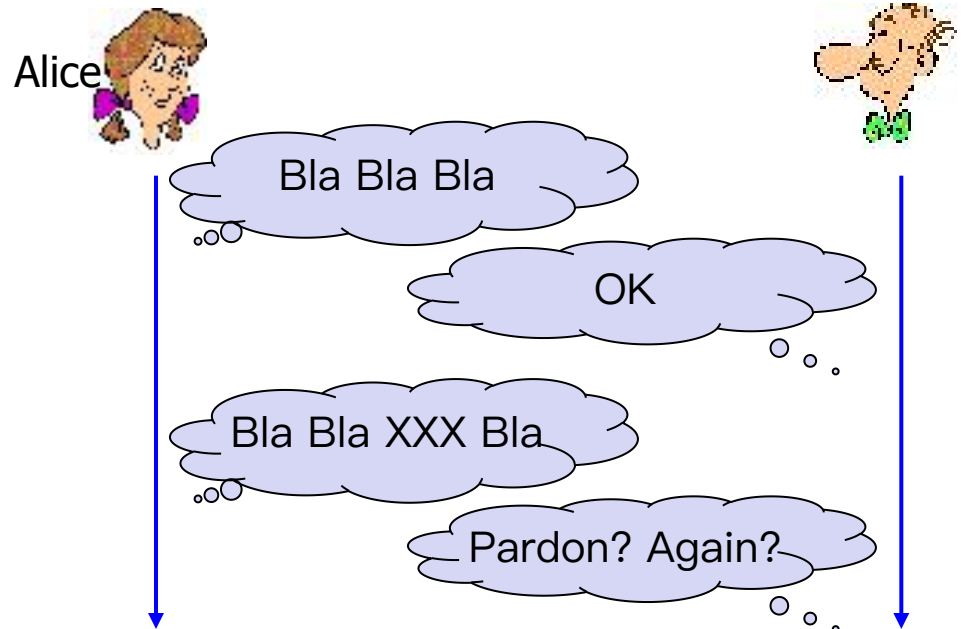
## Summary of Techniques

- Checksum
- Sequence number
- ACK packets
- Retransmission
- Timeout

### Stop and wait

Sender sends one packet,  
then waits for receiver  
response

### Limitations?

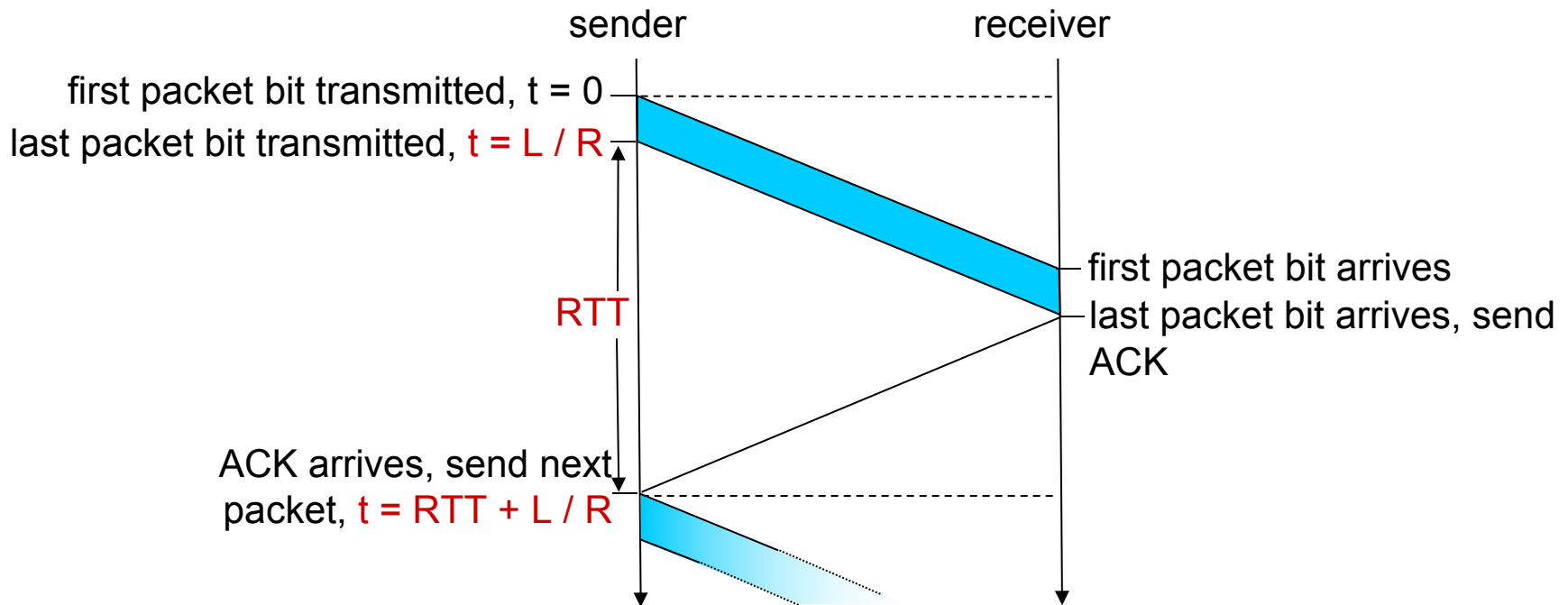


# Performance of rdt3.0

停等协议 (Stop-and-Wait Protocol)

必须等待ACK协议才会发下一个包

- ❖ rdt3.0 is correct, but performance is bad
- ❖ e.g.: link rate  $R=1$  Gbps, prop. delay  $T_{pd}=15$  ms, packet length  $L=8000$  bit



- Calculate **utilization**  $U_{\text{sender}}$ : **fraction of time sender busy sending**

# Performance of rdt3.0

- ❖ link rate  $R=1$  Gbps, prop. delay  $T_{pd}=15$  ms, packet length  $L=8000$  bit

$$D_{trans} = t = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \text{ microseconds}$$

- **utilization**  $U_{sender}$ : fraction of time sender busy sending

$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{.008}{30.008} = 0.00027$$

- RTT=30 msec, 1KB pkt every 30 msec:  
33kB/sec throughput over 1 Gbps link
- ❖ network protocol limits use of physical resources!

# Pipelined protocols

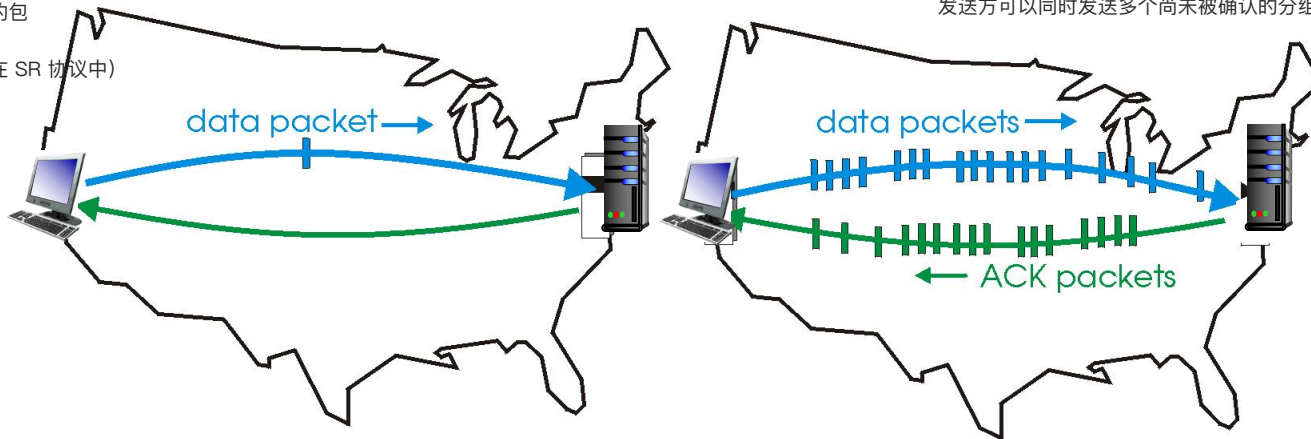
**pipelining:** sender allows multiple, “in-flight”, yet-to-be-acknowledged pkts

- range of sequence numbers must be increased
- **buffering** at sender and/or receiver

发送端：保存已发送但未确认的包

接收端：保存乱序到达的包（在 SR 协议中）

发送方可以同时发送多个尚未被确认的分组，不必等待每个 ACK 才继续发



(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

❖ two generic forms of pipelined protocols: *go-Back-N*, *selective repeat*

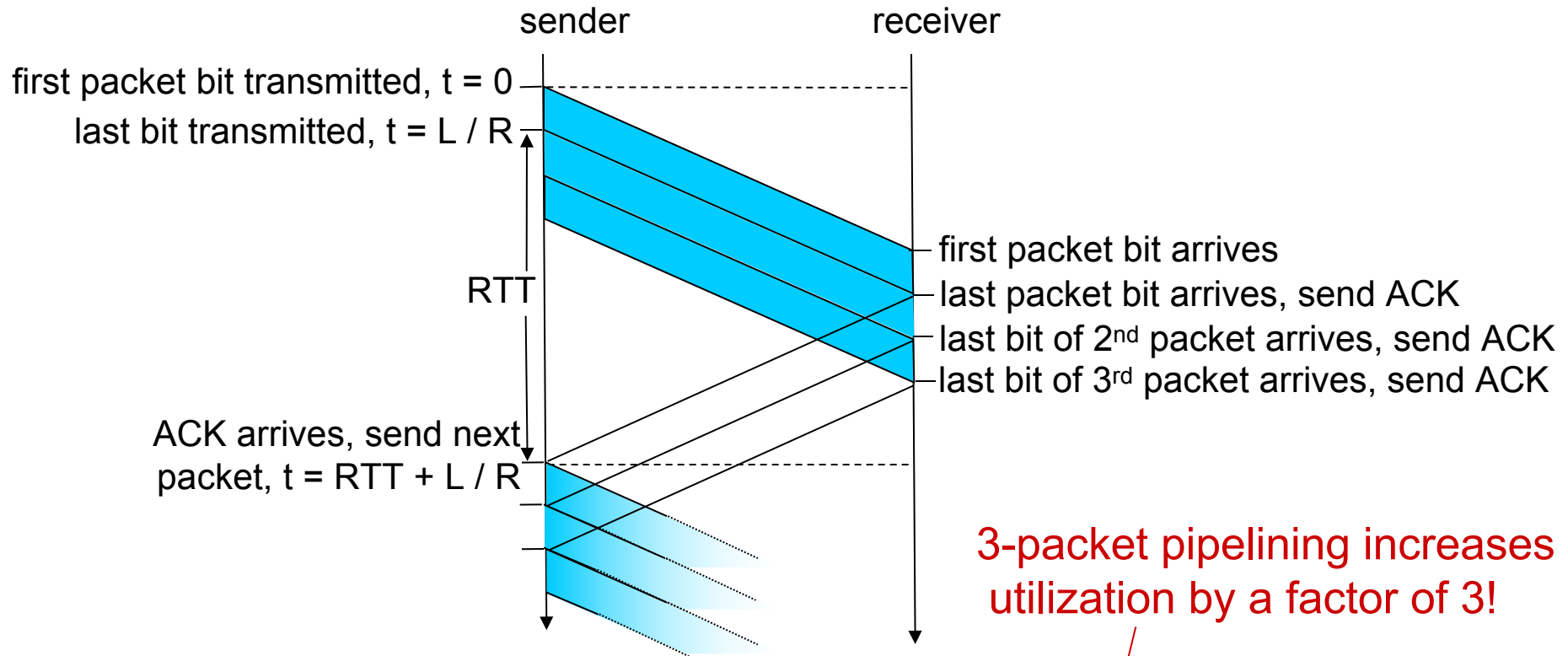
Go-Back-N (GBN)

如果出现错误，重传从错误分组开始之后的所有分组。

Selective Repeat (SR)

只重传出错的分组，效率更高但复杂度也更高。

# Pipelining: increased utilization



3-packet pipelining increases utilization by a factor of 3!

$$U_{\text{sender}} = \frac{3L / R}{RTT + L / R} = \frac{0.024}{30.008} = 0.00081$$

# Pipelined Protocols

## ❖ Go-Back-N

- Timer for the oldest unACKed packet
- Cumulative ACK
- Retransmit all packets in the window

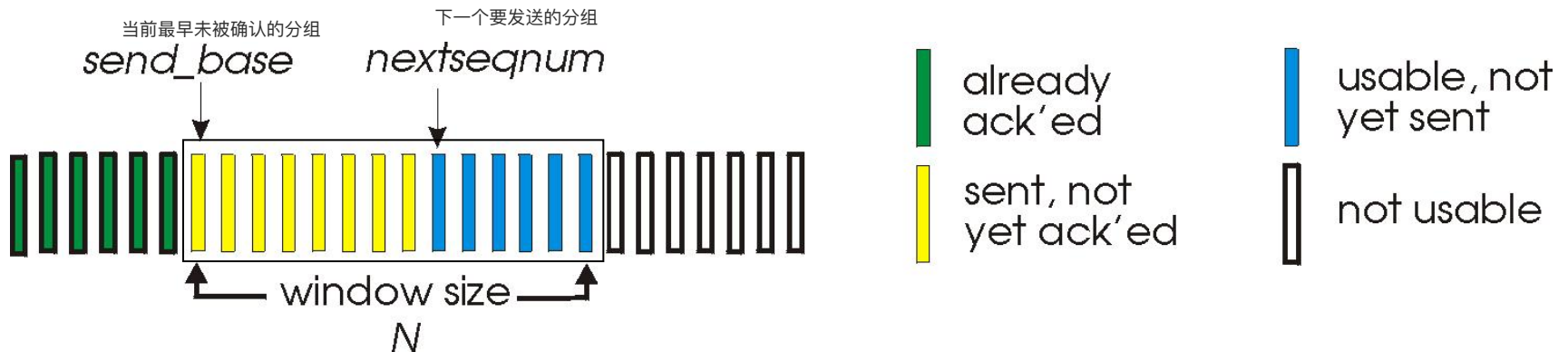
## ❖ Selective repeat

- Timer for each packet in window
- Individual ACK for each correctly received packets
- Retransmit only those packets that might be lost or corrupted

# Go-Back-N: Sender

- ❖  $k$ -bit seq # in pkt header (not 0 or 1):  $[0, 2^k - 1]$
- ❖ At most  $N$  pkts in flight: window size =  $N$ , ( $N$  consecutive unacked pkts allowed)

一次最多允许  $N$  个未确认的分组在途 (in-flight)，这个  $N$  就是 窗口大小 (Window Size)。



**Sender:** When `rdt_send()` is called from above,

- window is not full: a packet is sent, variables are updated.
- window is full: simply returns the data back to the upper layer

无法发送，必须等待 ACK (即上层“阻塞”)。

A **timer** for the **oldest** transmitted but not yet ACKed packet

只对 最早未确认的分组 (oldest unACKed packet) 启动定时器。  
如果超时，则 重传该分组及其后的所有分组 (“Go-Back”)。

# Go-Back-N: Receiver and Timeout

**Receiver:** Receipt of an ACK.

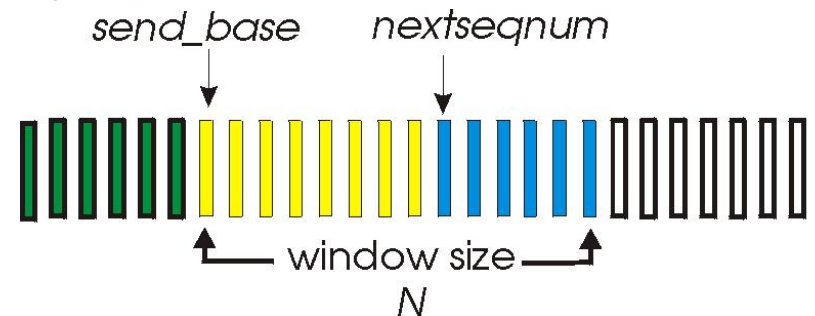
- **Cumulative acknowledgment (ACK)**
- **ACK( $n$ ):** all packets with a sequence # up to and including  $n$  have been correctly received at the receiver
- Expect  $n$  and receive  $n$ : ACK( $n$ )
- Expect  $n$  and receive others: previous ACK; discard packet

期望收到编号  $n$  的包：若收到了，发送 ACK( $n$ )。

若收到乱序包（非  $n$ ）：丢弃该包，重发上一个 ACK。  
(即 Go-Back-N 不缓存乱序数据)

**Sender:**

- **timeout** occurs: resends all packets in the window;
- ACK( $n$ ): slide window; restart timer



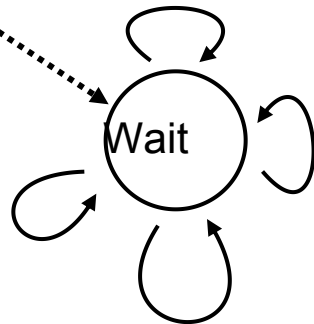


# GBN: sender extended FSM

rdt\_send(data)

```

if (nextseqnum < base+N) {
    sndpkt[nextseqnum] = make_pkt(nextseqnum,data,chksum)
    udt_send(sndpkt[nextseqnum])
    if (base == nextseqnum)
        start_timer
    nextseqnum++
}
else
    refuse_data(data)
    
```



rdt\_rcv(rcvpkt)  
&& corrupt(rcvpkt)

∧

rdt\_rcv(rcvpkt) &&  
notcorrupt(rcvpkt)

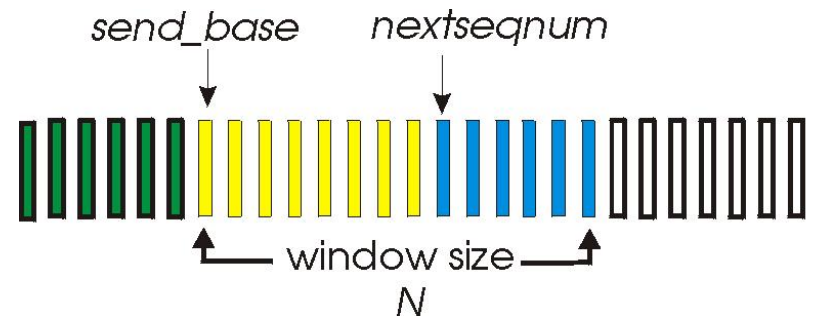
```

base = getacknum(rcvpkt)+1
If (base == nextseqnum)
    stop_timer
else
    start_timer
    
```

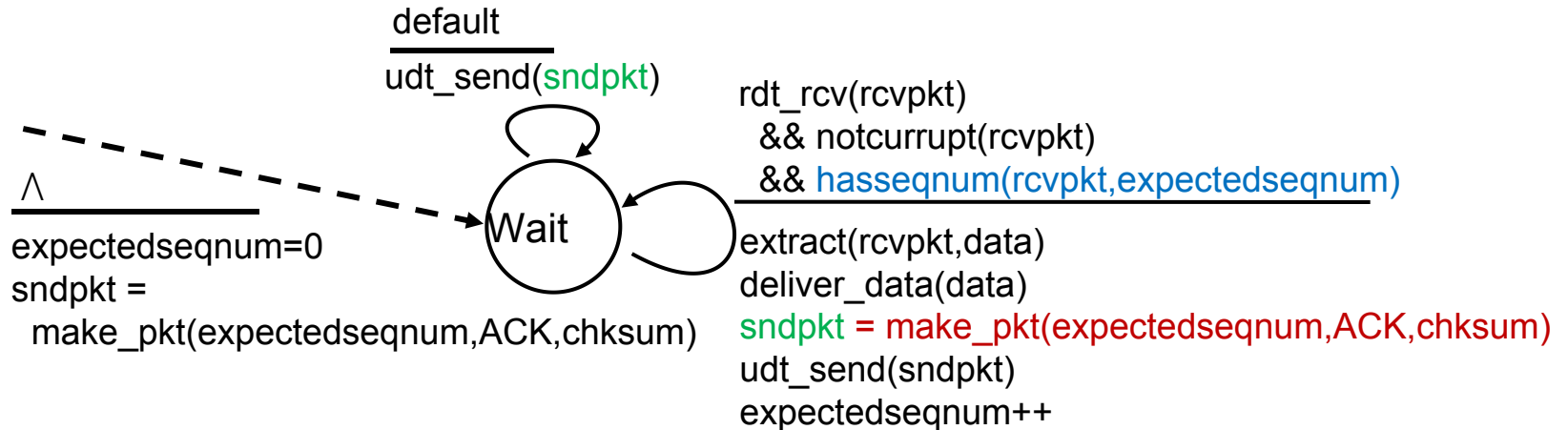
timeout

```

start_timer
udt_send(sndpkt[base])
udt_send(sndpkt[base+1])
...
udt_send(sndpkt[nextseqnum-1])
    
```



# GBN: receiver extended FSM



Cumulative ACK: always send ACK for correctly-received pkt with highest *in-order* seq #

- may generate duplicate ACKs

Go-Back-N 通过滑动窗口提高发送效率，但因为接收方不缓存乱序包，一旦丢包，发送方要“回退”并重传后续所有包——这就是它的名字“Go Back N”的由来

Out-of-order pkt:

- discard (don't buffer): *no receiver buffering!*
- re-ACK pkt with highest in-order seq #

# Go-Back-N Recall

sender window (N=4)

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

sender

send pkt0

send pkt1

send pkt2

send pkt3

(wait)

rcv ack0, send pkt4

rcv ack1, send pkt5

ignore duplicate ACK



*pkt 2 timeout*

send pkt2

send pkt3

send pkt4

send pkt5

receiver

No buffer,  
Cumulative ACK

receive pkt0, send ack0

receive pkt1, send ack1

receive pkt3, discard,  
(re)send **ack1**

receive pkt4, discard,  
(re)send **ack1**

receive pkt5, discard,  
(re)send **ack1**

rcv pkt2, deliver, send ack2

rcv pkt3, deliver, send ack3

rcv pkt4, deliver, send ack4

rcv pkt5, deliver, send ack5

Retransmit all pkts  
upon pkt loss or error

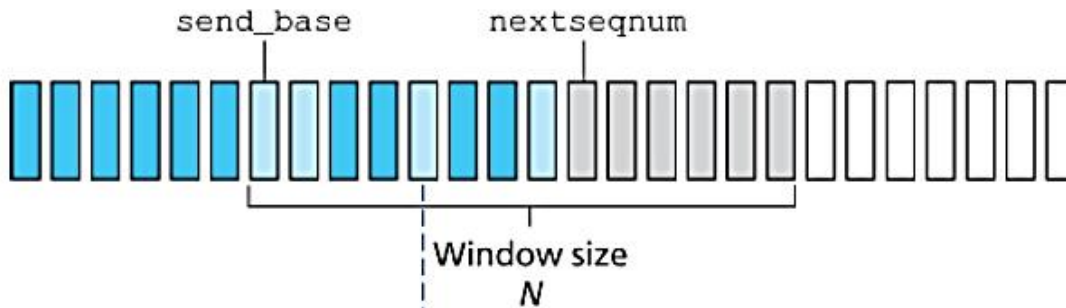
# Pipelined Protocols

- ❖ Go-Back-N
  - Timer for the oldest unACKed packet
  - Cumulative ACK
  - Retransmit all packets in the window
- ❖ Selective repeat
  - Timer for each packet in window
  - Individual ACK for each correctly received packets
  - Retransmit only those packets that might be lost or corrupted

# Selective repeat

- ❖ receiver *individually* acknowledges all correctly received pkts Selective Repeat 即使收到了乱序的数据包，只要是正确的，都会单独确认
  - buffers pkts, as needed, for eventual in-order delivery to upper layer
  - Receiver needs to keep track of the out-of-packets
- ❖ sender only resends pkts for which ACK not received 接收方会保存乱序到达但正确的数据包，直到所有缺失的、序列号更小的数据包到达，才能按序向上层应用交付
  - sender timer for each unACKed pkt

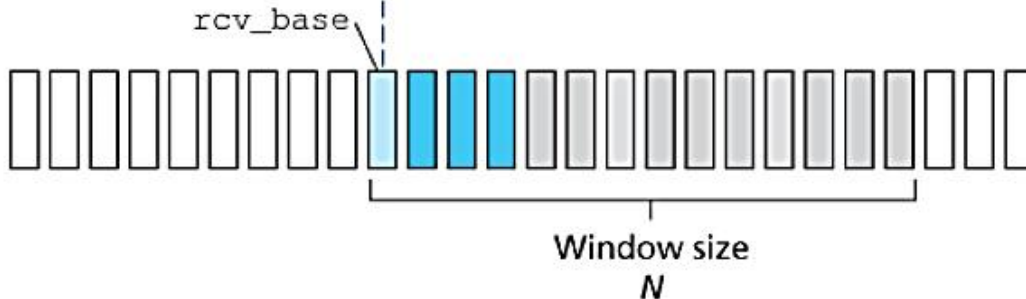
# Selective repeat: sender, receiver windows



a. Sender view of sequence numbers

Key:

|                                                                                     |                     |                                                                                     |                      |
|-------------------------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------|----------------------|
|  | Already ACK'd       |  | Usable, not yet sent |
|  | Sent, not yet ACK'd |  | Not usable           |



b. Receiver view of sequence numbers

Key:

|                                                                                       |                                           |                                                                                       |                            |
|---------------------------------------------------------------------------------------|-------------------------------------------|---------------------------------------------------------------------------------------|----------------------------|
|   | Out of order (buffered) but already ACK'd |   | Acceptable (within window) |
|  | Expected, not yet received                |  | Not usable                 |

# Selective repeat

## sender

### data from above:

- ❖ if next available seq # in window, send pkt

### timeout( $n$ ):

- ❖ resend pkt  $n$ , restart timer

### ACK( $n$ ) in $[\text{sendbase}, \text{sendbase}+N]$ :

- ❖ mark pkt  $n$  as received
- ❖ if  $n$  smallest unACKed pkt, advance window base to next unACKed seq #

## receiver

### pkt $n$ in $[\text{rcvbase}, \text{rcvbase}+N-1]$

- ❖ send ACK( $n$ )
- ❖ out-of-order: buffer
- ❖ in-order: deliver (also deliver buffered, in-order pkts), advance window to next not-yet-received pkt

### pkt $n$ in $[\text{rcvbase}-N, \text{rcvbase}-1]$

- ❖ ACK( $n$ ) 如果数据包是已经交付过的包, receiver还是会ACK防止 sender误判

### otherwise:

- ❖ ignore

# Selective repeat

sender window (N=4)

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

0 1 2 3 4 5 6 7 8

sender

send pkt0

send pkt1

send pkt2

send pkt3

(wait)

rcv ack0, send pkt4

rcv ack1, send pkt5

record ack3 arrived



*pkt 2 timeout*

send pkt2

record ack4 arrived

record ack5 arrived

receiver

have buffer,  
individual ACK

receive pkt0, send ack0

receive pkt1, send ack1

receive pkt3, buffer,  
send ack3

receive pkt4, buffer,  
send ack4

receive pkt5, buffer,  
send ack5

rcv pkt2; deliver pkt2,  
pkt3, pkt4, pkt5; send ack2

*X loss*

*what happens when ack2 arrives?*

Only retransmit the  
unacked pkt (SR)



# Selective repeat: dilemma

Example:

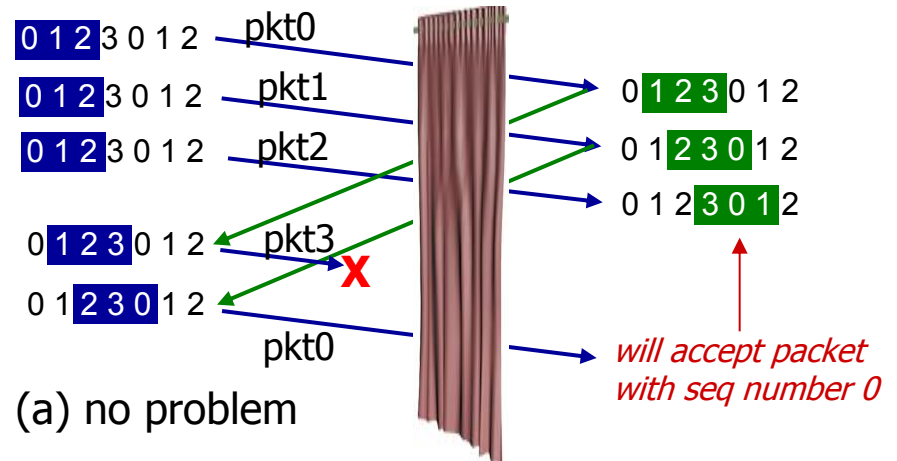
- ❖ seq #'s: 0, 1, 2, 3
- ❖ window size=3
- ❖ receiver sees no difference in two scenarios!
- ❖ duplicate data accepted as new in (b)

**Q:** what relationship between seq # size and window size to avoid problem in (b)?

The window size must be less than or equal to half the size of the sequence number space for SR protocols.

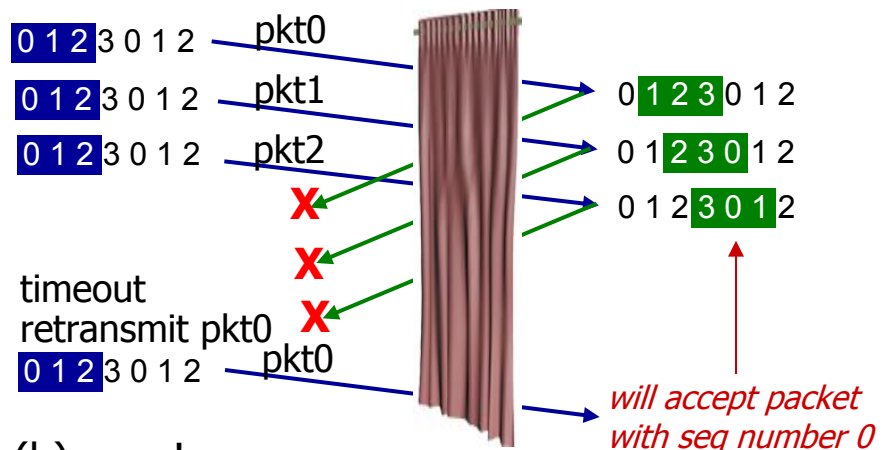
sender window (after receipt)

receiver window (after receipt)



(a) no problem

*receiver can't see sender side.  
receiver behavior identical in both cases!  
something's (very) wrong!*



(b) oops!

# GBN and SR comparison

## Go-back-N:

- ❖ sender can have up to  $N$  unacked packets in pipeline
- ❖ receiver only sends *cumulative ack*
  - doesn't ack packet if there's a gap
- ❖ sender has timer for oldest unacked packet
  - when timer expires, retransmit *all* unacked packets

## Selective Repeat:

- ❖ sender can have up to  $N$  unack'ed packets in pipeline
- ❖ rcvr sends *individual ack* for each packet
- ❖ sender maintains timer for each unacked packet
  - when timer expires, retransmit only that unacked packet

# Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 **connection-oriented transport: TCP**

- segment structure, RTT measurement
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

# TCP: Overview

RFCs: 793, 1122, 1323, 2018, 2581

TCP 连接的“状态”和“存储”只在通信两端的主机 (sender/receiver) 里维护，而中间的网络设备 (例如路由器、交换机) 不会为某一条 TCP 连接专门分配、保存“连接变量/状态”或“专用缓冲区”

## ❖ point-to-point:

end to end  
TCP的可靠性都是端系统负责的

- one sender, one receiver
- No buffers or variables are allocated to network elements between hosts

## ❖ reliable, in-order byte stream:

TCP像是可靠的管道在传连续的字节  
所以TCP没有message boundary  
所以需要在每条消息的边界，自己在应用层加分隔符

- no “message boundaries”
- Seq # and Ack # are in unit of byte, rather than pkt

接收方交给应用的数据，顺序一定和发送方写入的顺序一致

就算网络里发生乱序 (后发的包先到)，TCP 会在接收端先缓存，等缺的那块来了，再按顺序交付给应用。

## ❖ pipelined:

- TCP congestion and flow control set window size

## ❖ full duplex data:

在同一条 TCP 连接里，通信双方可以同时发送数据和接收数据，也就是双向同时传输  
包含两个独立的数据流方向

- bi-directional data flow in same connection
- MSS: maximum segment size

## ❖ connection-oriented:

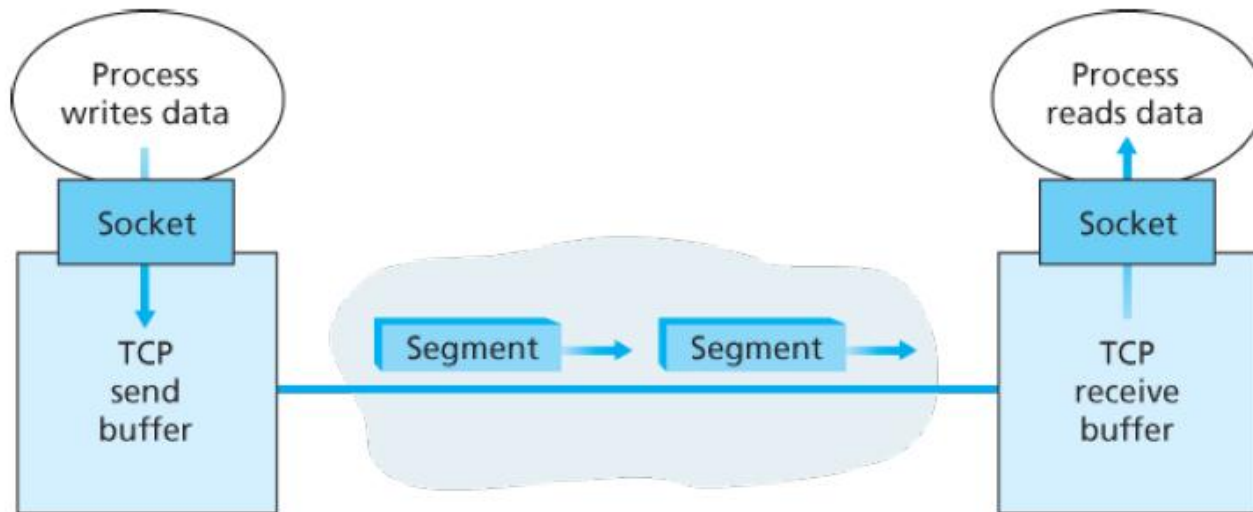
- handshaking (exchange of control msgs) initiates sender and receiver state before data exchange

## ❖ flow controlled:

- sender will not overwhelm receiver

# TCP: Overview

RFCs: 793, 1122, 1323, 2018, 2581

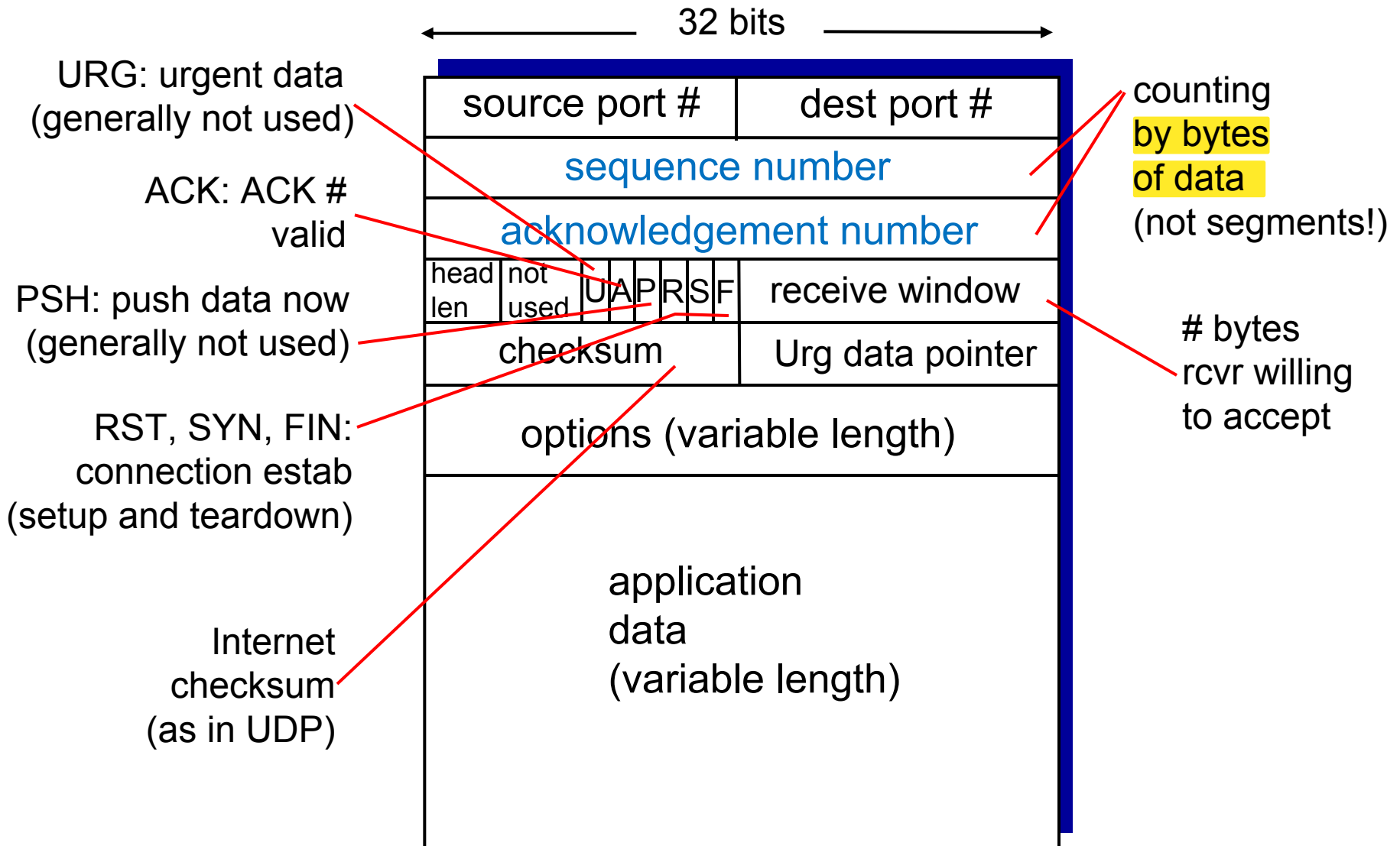


- TCP connection
- TCP grab chunks of data from the sender buffer
  - MSS: maximum **segment size**, typically 1460 bytes
  - MTU: maximum transmission unit (link-layer frame), typically 1500 bytes
    - Application data + **TCP/IP header (typically 40 bytes)**
- TCP receives a segment at the other end, place it in receiver buffer
- application reads the stream from the **receive buffer**

# TCP Reliable Data Transfer

- ❖ Segment structure
  - Segment format
  - Seq. number and ACKs
  - An example
- ❖ Round-trip time estimation
- ❖ Reliable data transfer
- ❖ Flow control
- ❖ Control management

# TCP segment structure



# TCP seq. numbers, ACKs

只能看到连续bytes, 而没有消息边界

TCP views data as **an unstructured, but ordered, stream of bytes.**

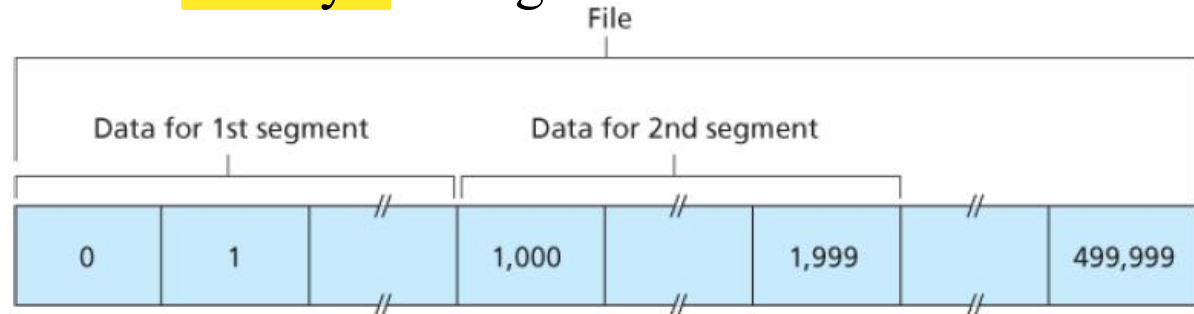
- Sequence numbers are over the **stream** of transmitted bytes and **not** over the series of transmitted **segments**

每个字节流都有一个唯一的“序列号”，指的是数据段中第一个字节的编号。这个编号用于标识数据流中每个字节的顺序

## sequence numbers:

- byte stream “number” of **first byte** in segment's data

某一段seq=0, 长度=1000, 那么就覆盖0-999



## acknowledgements:

- seq # of **next byte expected** from other side

- E.g., receiver has received bytes numbered 0 through 535 and 900 through 1000; then, acknowledgement number is 536.

- cumulative ACK

**Q:** how receiver handles out-of-order segments

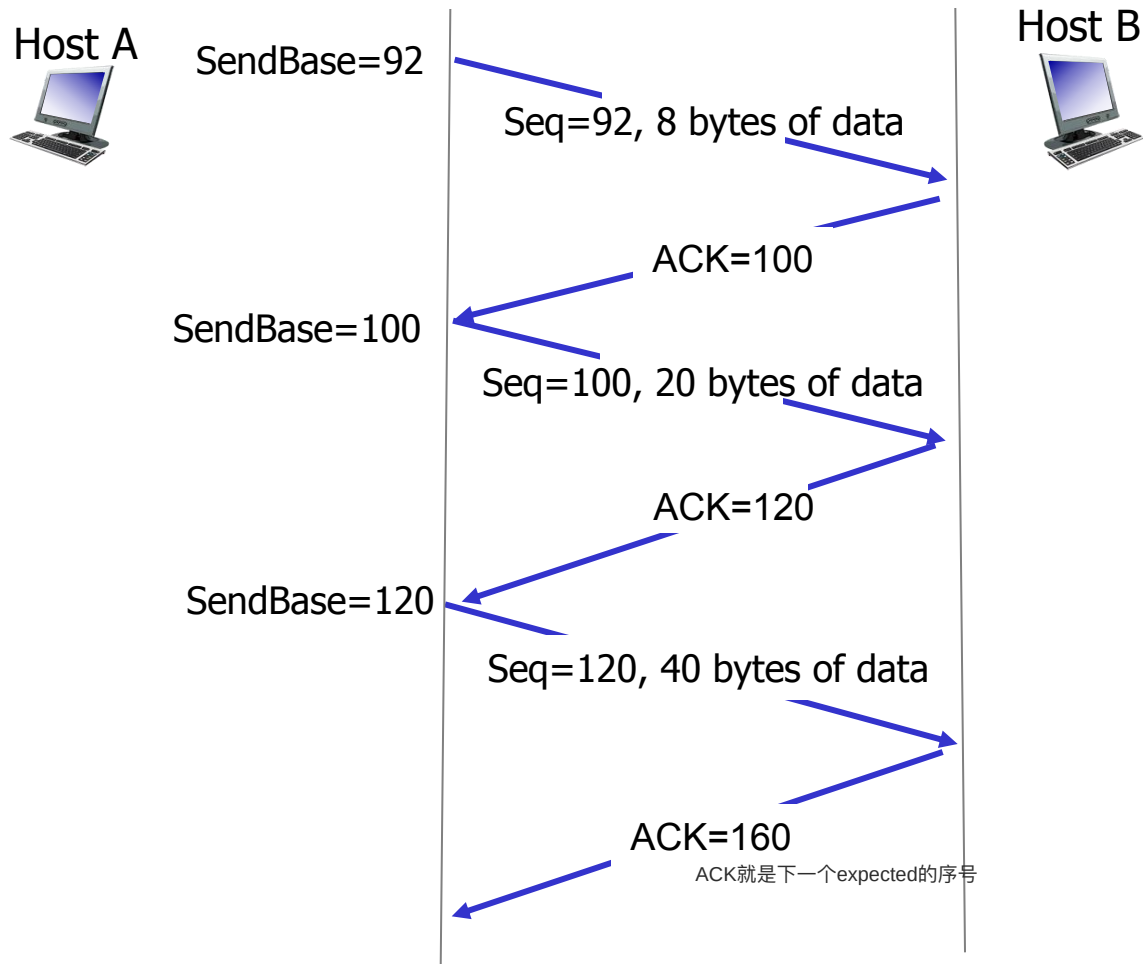
- A:** TCP spec doesn't say, - up to implementor

Initial sequence number is randomly chosen

建立连接时 (3次握手), 双方选一个初始序列号ISN, 防止旧连接遗留的包混进新连接, 提高安全性 (更难猜序列号)



# TCP Example



# Telnet Case Study

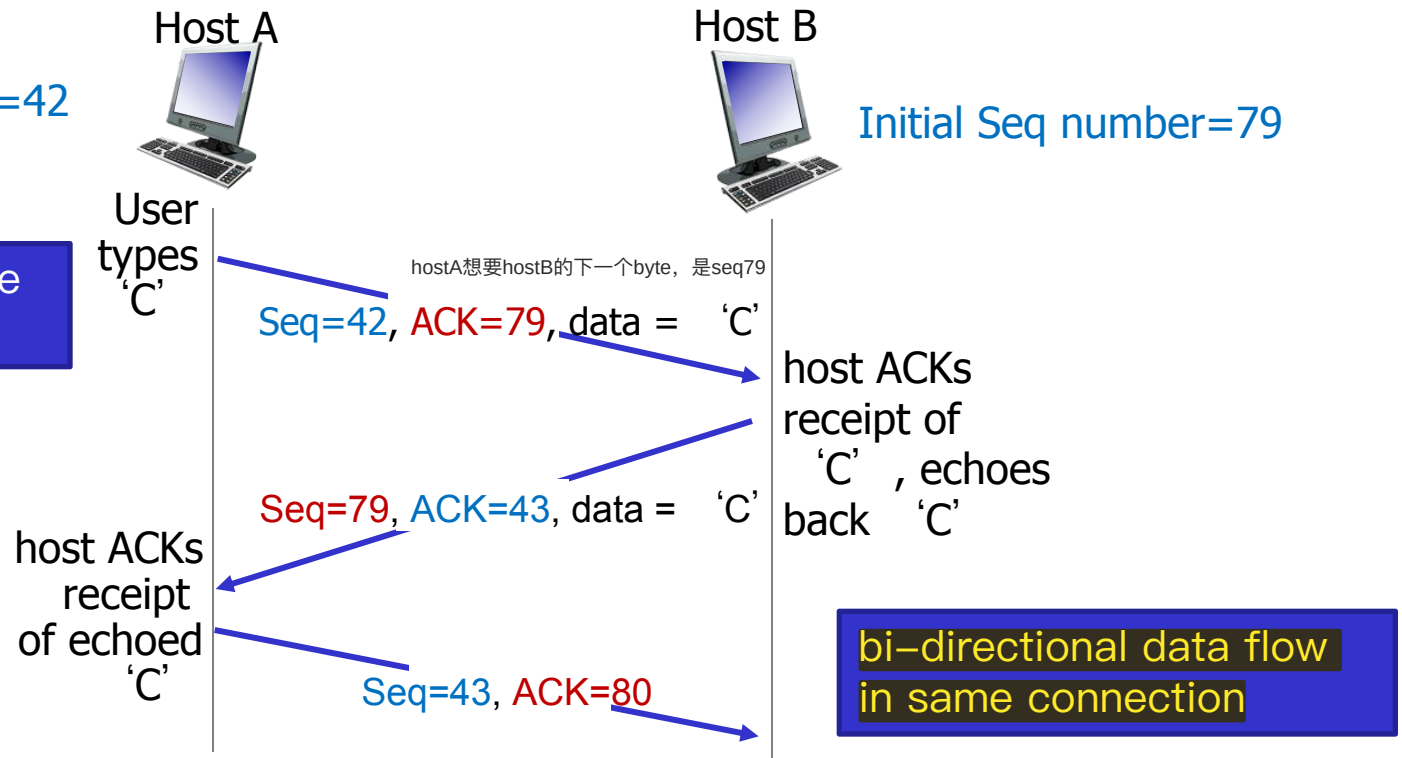
- User types a character at host A, and host A sends the character to host B
- Host B sends back a copy of the character
- Host A displays the character on user's screen

Initial Seq number=42

Initial Seq number=79

ACK # is the next byte expected from other

这里C就是



实际上这并不意味着序列号变小, 而是数据在两个方向流动时的不同序列号的使用。在一个TCP连接中, 序列号通常是递增的, 但是每一端都维护自己的独立序列号 (即发送方的序列号和接收方的确认号)

# TCP Reliable Data Transfer

- ❖ Segment structure
- ❖ Round-trip time estimation
- ❖ Reliable data transfer
- ❖ Flow control
- ❖ Control management

# TCP round trip time, timeout

Q: How to set TCP timeout value?

- ❖ longer than RTT
  - but RTT **varies**
- ❖ *too short*: premature timeout, unnecessary retransmissions
- ❖ *too long*: slow reaction to segment loss

Q: How to estimate RTT?

- ❖ **SampleRTT**: measured time from segment transmission until ACK receipt
  - ignore retransmissions
- ❖ **SampleRTT** will vary, want estimated RTT “smoother”
  - average several recent measurements, not just current **SampleRTT**

# TCP round trip time, timeout

指数加权移动平均

(EWMA)

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

sampleRTT一次测量值会抖动，但是estimatedRTT更平滑

- ❖ exponential weighted moving average
- ❖ influence of past sample decreases exponentially fast
- ❖ typical value:  $\alpha = 0.125$

## Example:

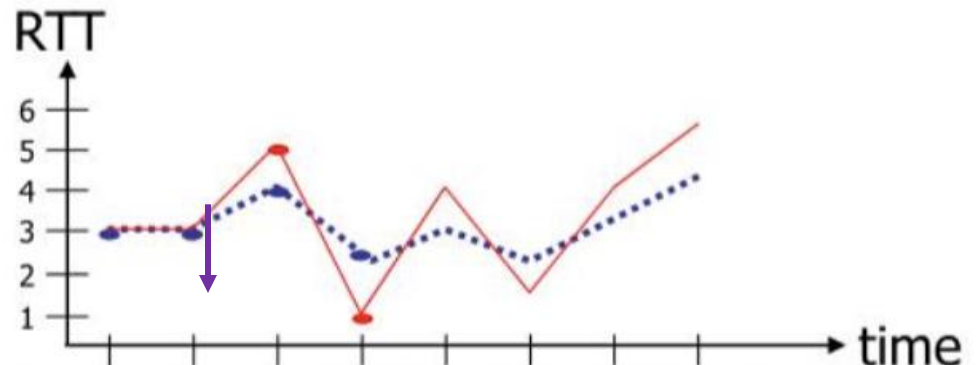
Suppose  $\alpha = 0.5$

EstimatedRTT = 3

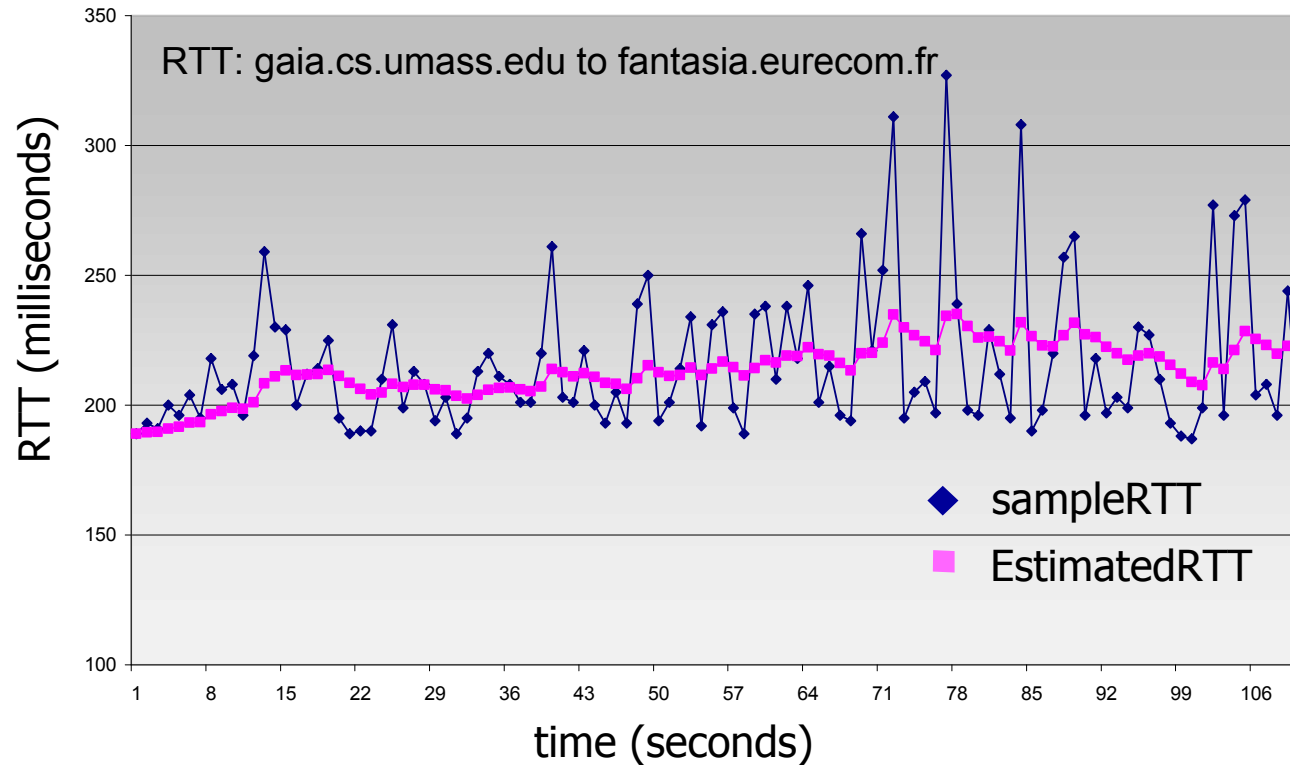
2) EstimatedRTT =  $.5 * 3 + .5 * 3 = 3$

3) EstimatedRTT =  $.5 * 3 + .5 * 5 = 4$

4) EstimatedRTT =  $.5 * 4 + .5 * 1 = 2.5$



# TCP round trip time, timeout



# TCP round trip time, timeout

**Variability** of the RTT: how much SampleRTT deviates from EstimatedRTT:  
“RTT的波动”的平滑估计

$\text{DevRTT} = (1-\beta) * \text{DevRTT} +$  DevRTT用于衡量RTT的变化量

$\beta * |\text{SampleRTT} - \text{EstimatedRTT}|$

(typically,  $\beta = 0.25$ )

**TCP timeout interval:** EstimatedRTT plus “safety margin”  
large variation in EstimatedRTT -> larger safety margin

$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$



↑  
estimated RTT

↑  
“safety margin”

给一个比较保守的缓冲，避免网络只是暂时变慢就丢包

# TCP Reliable Data Transfer

- ❖ Segment structure
- ❖ Round-trip time estimation
- ❖ **Reliable data transfer**
- ❖ Flow control
- ❖ Control management



# TCP reliable data transfer

- ❖ TCP creates rdt service on top of IP's unreliable service
  - pipelined segments: window size, SendBase
  - **cumulative acks**
  - single retransmission timer
- ❖ retransmissions triggered by:
  - timeout events
  - duplicate acks

Let's initially consider simplified TCP sender:

- ignore duplicate acks
- ignore flow control, congestion control

# TCP sender events:

## *data rcvd from app:*

- ❖ create segment with seq #
- ❖ seq # is byte-stream number of first data byte in segment
- ❖ start timer if not already running
  - think of timer as for oldest unacked segment
  - expiration interval: **TimeoutInterval**

## *timeout:*

- ❖ retransmit segment that caused timeout
- ❖ restart timer

## *ack rcvd:*

- ❖ if ack acknowledges previously unacked segments
  - update what is known to be ACKed
  - start timer if there are still unacked segments

# TCP sender events:

```
NextSeqNum=InitialSeqNumber  
SendBase=InitialSeqNumber
```

```
loop (forever) {  
    switch(event)
```

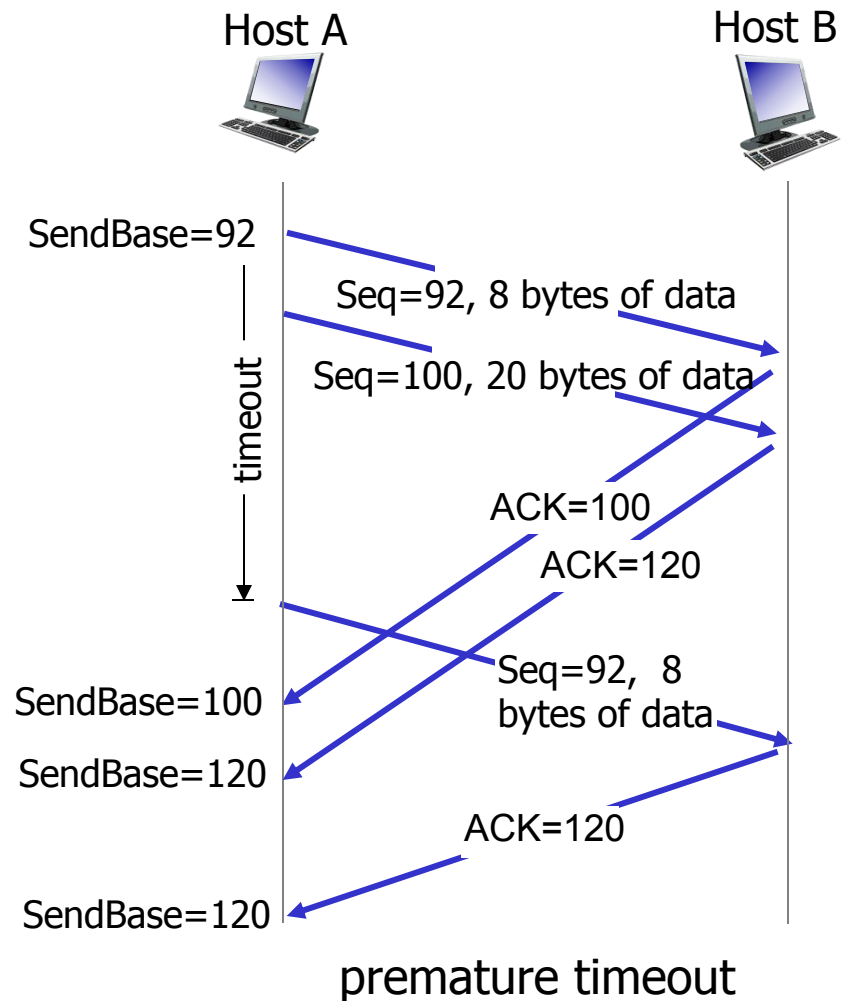
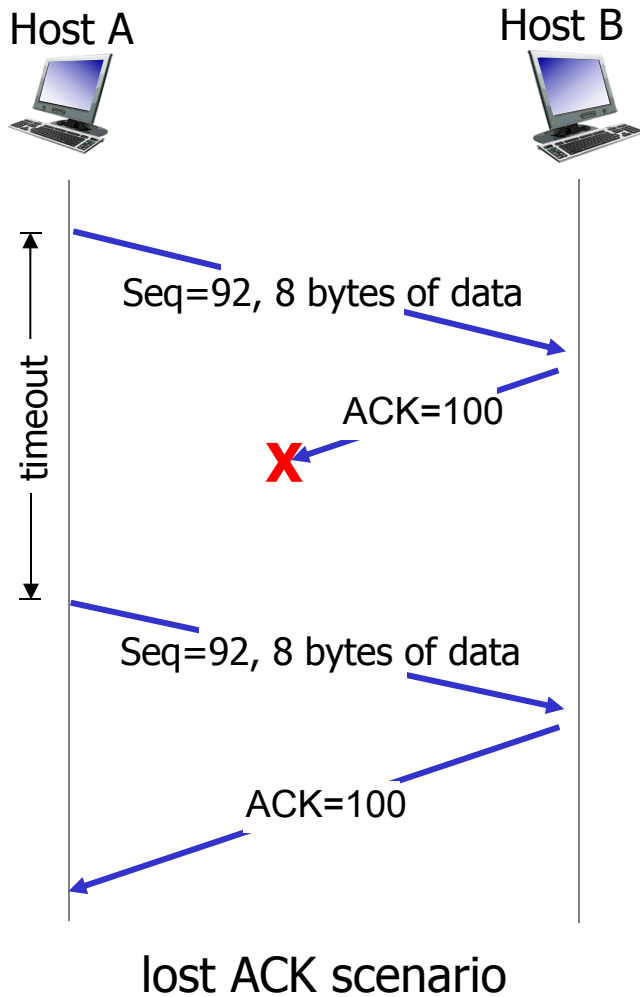
```
event: data received from application above  
    create TCP segment with sequence number NextSeqNum  
    if (timer currently not running)  
        start timer  
    pass segment to IP  
    NextSeqNum=NextSeqNum+length(data)  
    break;
```

```
event: timer timeout  
    retransmit not-yet-acknowledged segment with  
        smallest sequence number  
    start timer  
    break;
```

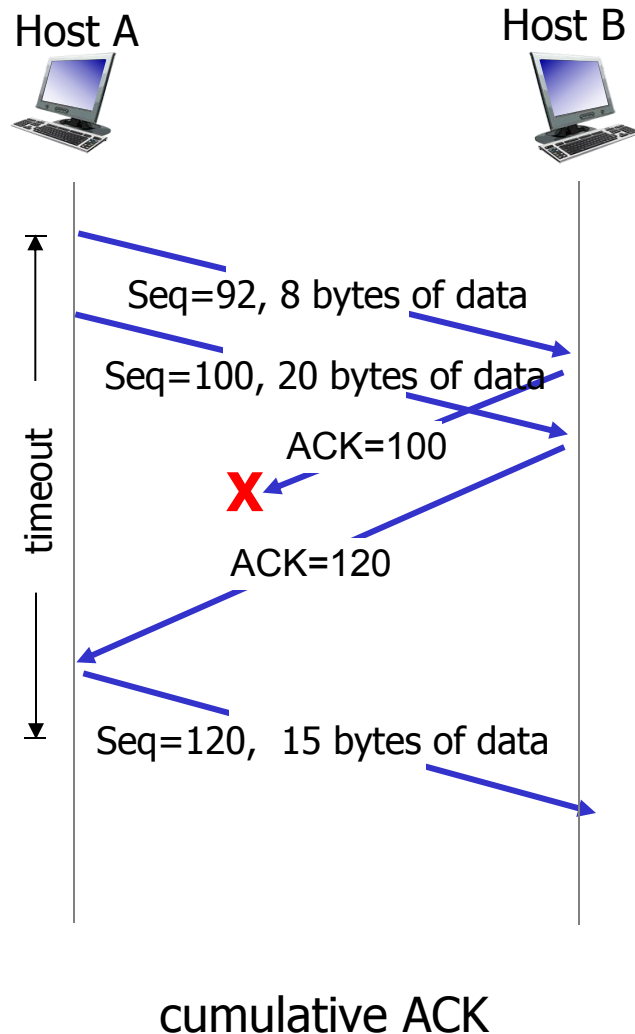
```
event: ACK received, with ACK field value of y  
    if (y > SendBase) {  
        SendBase=y  
        if (there are currently any not-yet-acknowledged segments)  
            start timer  
    }  
    break;
```

```
} /* end of loop forever */
```

# TCP: retransmission scenarios



# TCP: retransmission scenarios



# TCP receiver [RFC 1122, RFC 2581]

## *event at receiver*

## *TCP receiver action*

收到按序的ACK，并且之前的都ACK过

arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed

**delayed ACK.** Wait up to 500ms for next segment. If no next segment, send ACK

收到按序的ACK，但之前还有一个ACK没发

arrival of in-order segment with expected seq #. One other segment has ACK pending

immediately **send single cumulative ACK**, ACKing both in-order segments

收到乱序的segment，发现gap

arrival of out-of-order segment **higher-than-expect seq. #**. Gap detected

immediately **send duplicate ACK**, indicating seq. # of next expected byte

收到的segment填补了gap（部分或者全部）

arrival of segment that partially or completely **fills gap**

**immediate send ACK**, provided that segment starts at lower end of gap

# Double Timeout Interval

Each time TCP retransmits, it sets the next timeout interval to twice the previous value

- ❖ Suppose *TimeoutInterval* associated with the oldest not yet acknowledged segment is 0.75 sec.
- ❖ when the timer first expires, TCP will then retransmit this segment and set the new expiration time to 1.5 sec.
- ❖ If the timer expires again, TCP will again retransmit this segment, now setting the expiration time to 3.0 sec.

这种超时时间间隔的调整机制确保了TCP能够在遇到网络延迟或丢包时逐步增加等待时间，从而避免频繁的重传请求

当timer是由其他时间启动时（ACK或者应用层给了新数据），不会继续翻倍了

When the timer is started after either of the two other events (that is, data received from application above, and ACK received):

- ❖ the *TimeoutInterval* is derived from the most recent values of *EstimatedRTT* and *DevRTT*.

这个时候重新启用estimatedRTT+4DevRTT

Part of congestion control: being polite

# TCP fast retransmit

收到乱序段 → 立即回 duplicate ACK, ACK不前进

- ❖ time-out period often relatively long:
  - long delay before resending lost packet
- ❖ detect lost segments via duplicate ACKs.
  - sender often sends many segments back-to-back
  - if segment is lost, there will likely be many duplicate ACKs.

如果发送方：  
连续发了很多段  
其中 某一个中间段丢了  
后面的段陆续到达接收方

那么接收方就会：  
对每个乱序段都回 同一个 ACK  
发送方就会收到 很多 duplicate ACK

## *TCP fast retransmit*

if sender receives **3 duplicate ACKs** for same data

应该是4个, 3个重复的

( “triple duplicate ACKs” ), resend unacked segment with smallest seq #

- likely that unacked segment lost, so don't wait for timeout



# TCP fast retransmit

```
NextSeqNum=InitialSeqNumber
SendBase=InitialSeqNumber

loop (forever) {
    switch(event)

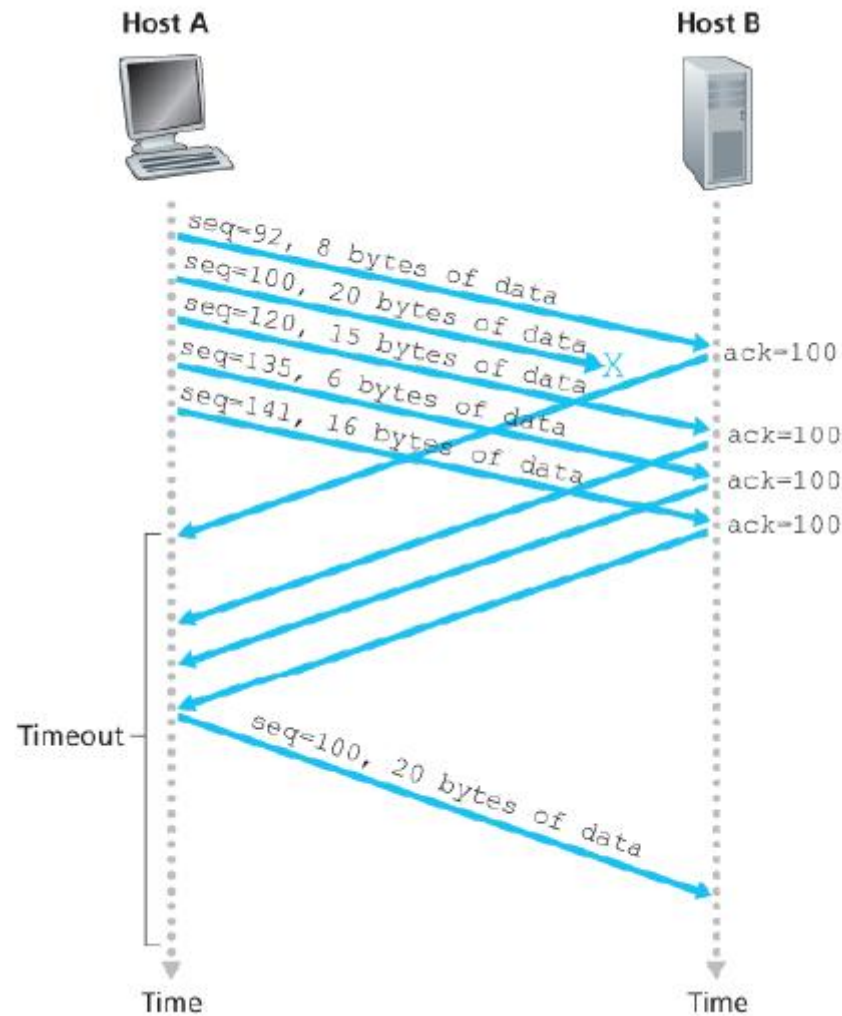
        event: data received from application
            create TCP segment with sequence
            if (timer currently not running)
                start timer
            pass segment to IP
            NextSeqNum=NextSeqNum+length(data)
            break;

        event: timer timeout
            retransmit not-yet-acknowledged segment ,
                smallest sequence number
            start timer
            break;

        event: ACK received, with ACK field value of y
            if (y > SendBase) {
                SendBase=y
                if (there are currently any not yet
                    acknowledged segments)
                    start timer
            }
            else { /* a duplicate ACK for already ACKed
                segment */
                increment number of duplicate ACKs
                received for y
                if (number of duplicate ACKs received
                    for y==3)
                    /* TCP fast retransmit */
                    resend segment with sequence number y
            }
            break;

    } /* end of loop forever */
```

# TCP fast retransmit



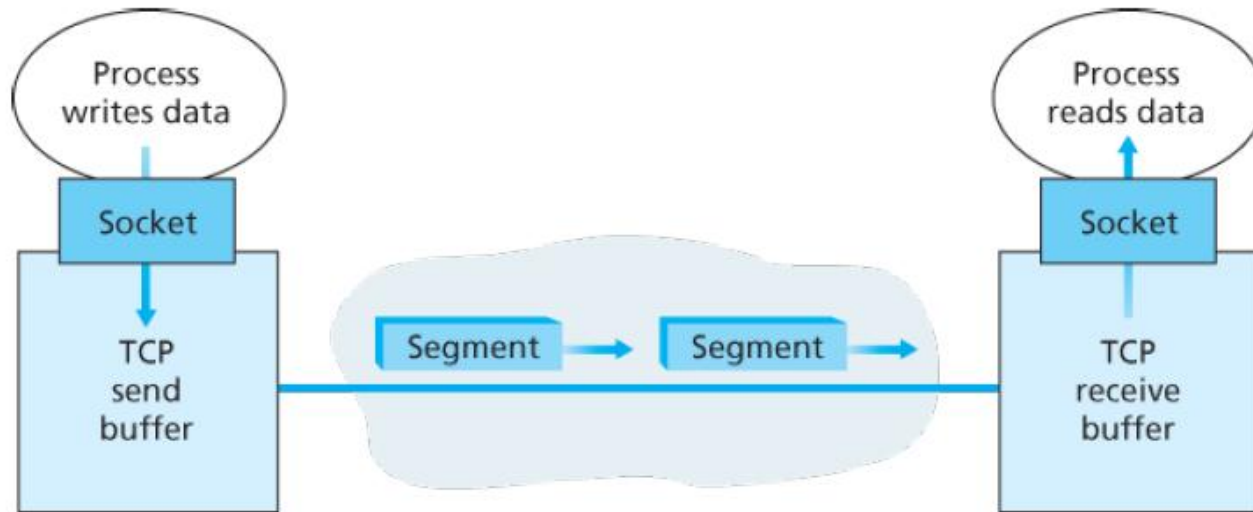
fast retransmit after sender receipt of triple duplicate ACK

# TCP Reliable Data Transfer

- ❖ Segment structure
- ❖ Round-trip time estimation
- ❖ Reliable data transfer
- ❖ **Flow control**
- ❖ Control management

# TCP: Overview

---



- TCP connection
- TCP grab chunks of data from the sender buffer
- TCP receives a segment at the other end, place it in receiver buffer
- application reads the stream from the receive buffer

# TCP flow control

application may  
remove data from  
TCP socket buffers ....

- TCP 接收方把数据先放进 接收缓冲区 (TCP receiver buffer)

- 应用进程再从这个 buffer 里慢慢 read()

- 应用可能很慢 (比如写磁盘、GUI 卡顿)

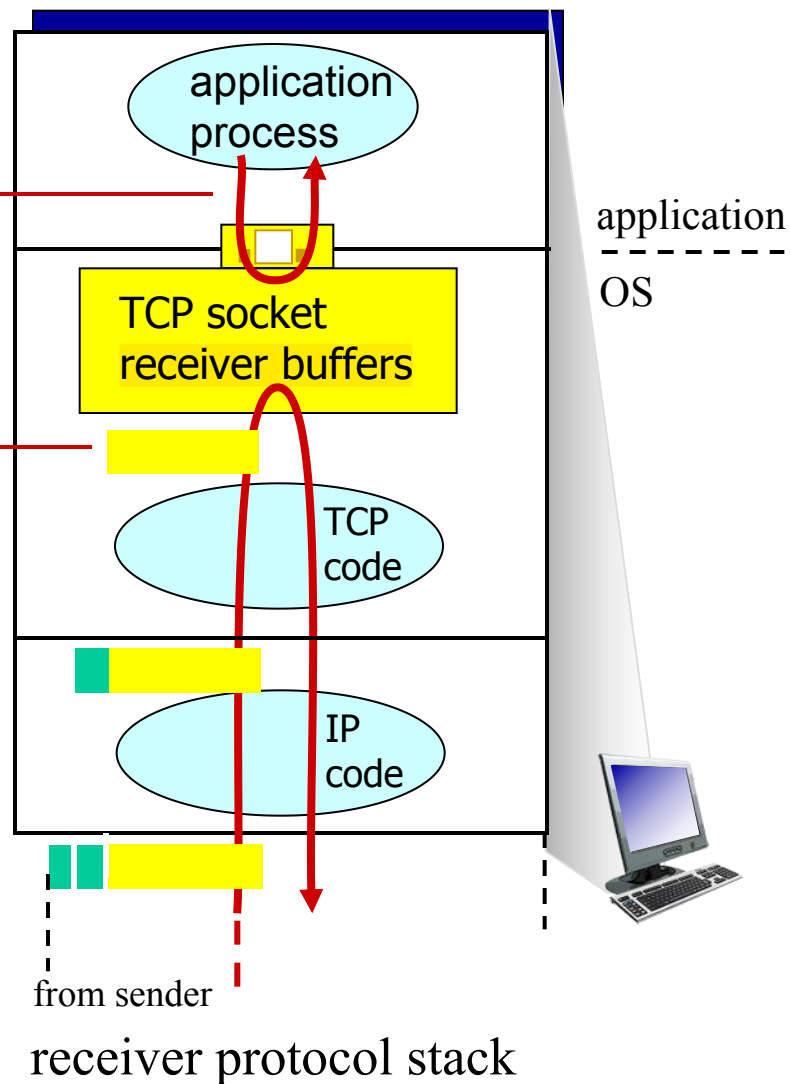
- 但 发送方可能很快

如果发送方不受限制地一直发:

- 接收缓冲区会被写满

... slower than TCP  
receiver is delivering  
(sender is sending)

*flow control* 接收方控制 sender  
Receiver controls sender, so  
sender won't overflow  
receiver's buffer by  
transmitting too much, too fast

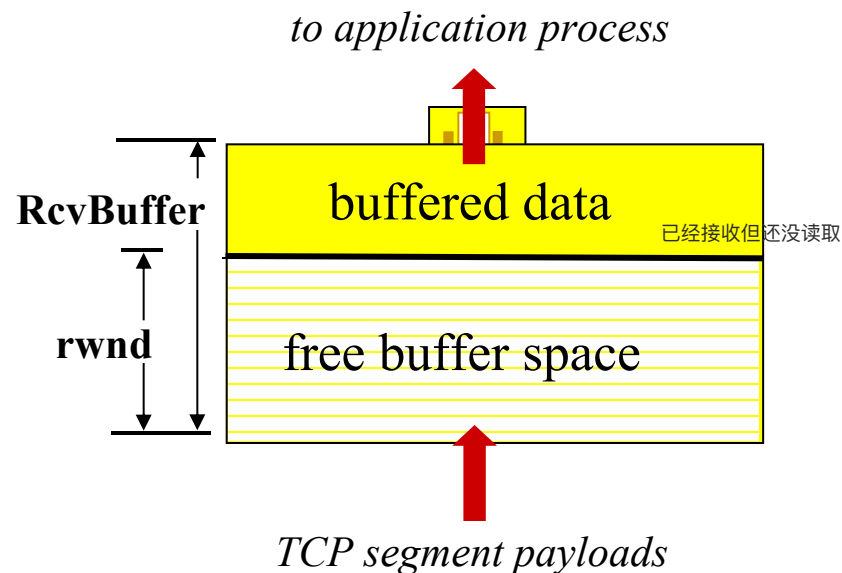


# TCP flow control

接收方通过在发送给发送方的TCP段头部包含rwnd（接收窗口）值，来通知发送方当前接收方的缓冲区中还有多少可用空间

**Receiver** “advertises” free buffer space by including **rwnd** value in TCP header of receiver-to-sender segments

- ❖ **RcvBuffer** size set via socket options (typical default is 4096 bytes)
- ❖ many operating systems autoadjust **RcvBuffer**

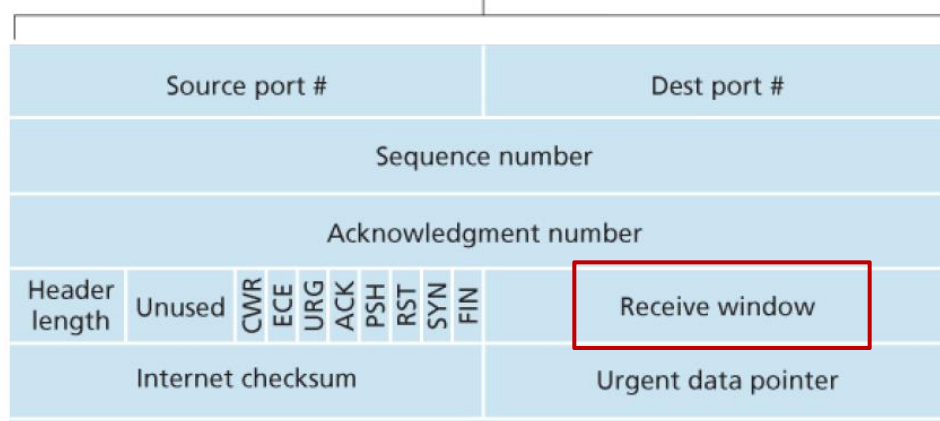


$$rwnd = RcvBuffer - [LastByteRcvd - LastByteRead]$$

RcvBuffer是接收缓冲区的总大小，LastByteRcvd是接收到的最后一个字节的序列号，LastByteRead是应用程序读取的最后一个字节的序列号

*receiver-side buffering*

发送方维持一个规则：  
未被ACK的数据《=rwnd

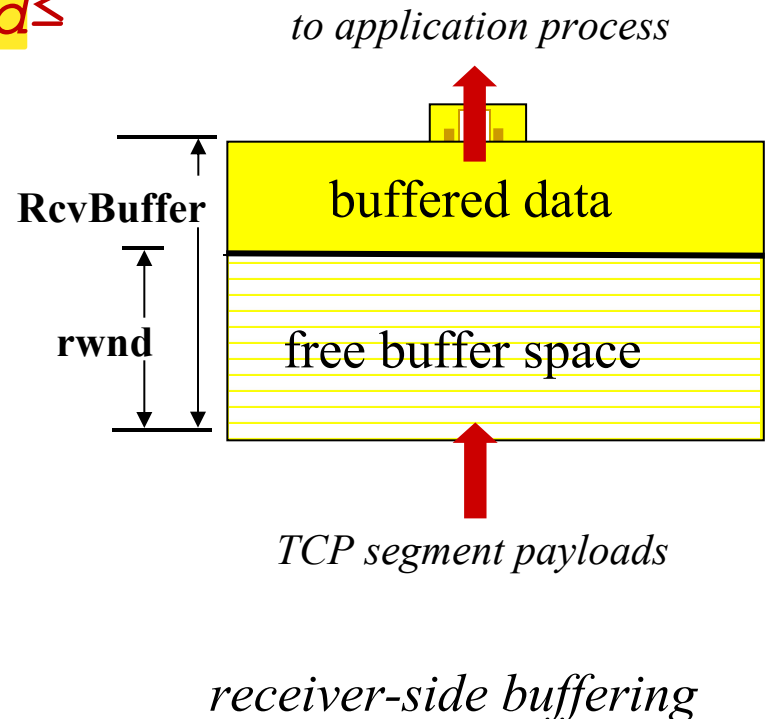


# TCP flow control

**Sender** limits amount of  
unacked ( “in-flight” ) data to  
receiver's **rwnd** value

$LastByteSent - LastByteAcked \leq$   
 $rwnd$

Guarantees receive buffer will  
not overflow



# TCP Reliable Data Transfer

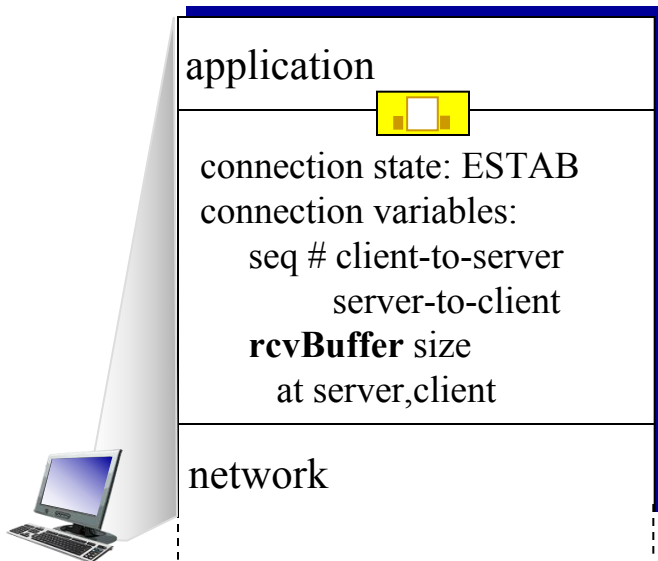
- ❖ Segment structure
- ❖ Round-trip time estimation
- ❖ Reliable data transfer
- ❖ Flow control
- ❖ Control management



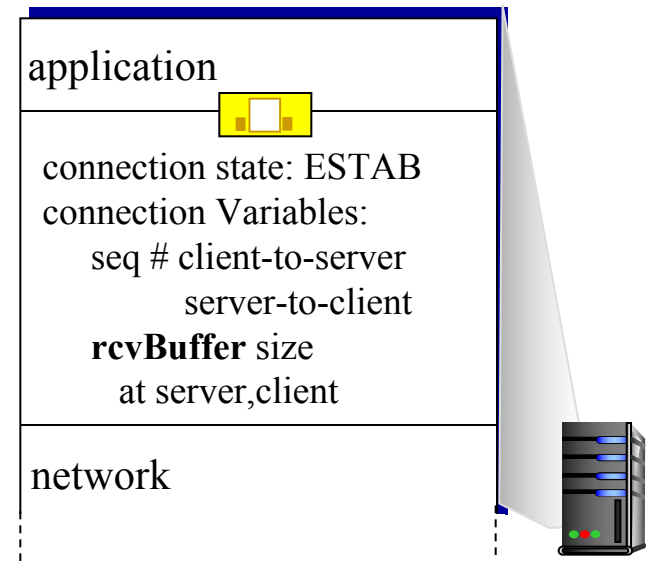
# Connection Management

before exchanging data, sender/receiver “handshake” :

- ❖ agree to establish connection (each knowing the other willing to establish connection)
- ❖ agree on connection parameters



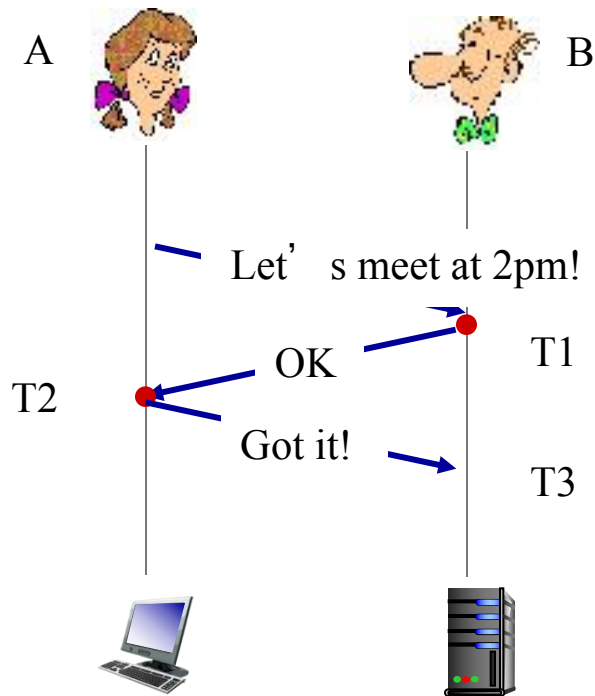
```
clientSocket = socket(AF_INET, SOCK_STREAM);  
clientSocket.connect((hostname,port number));
```



```
connectionSocket = welcomeSocket.accept();
```

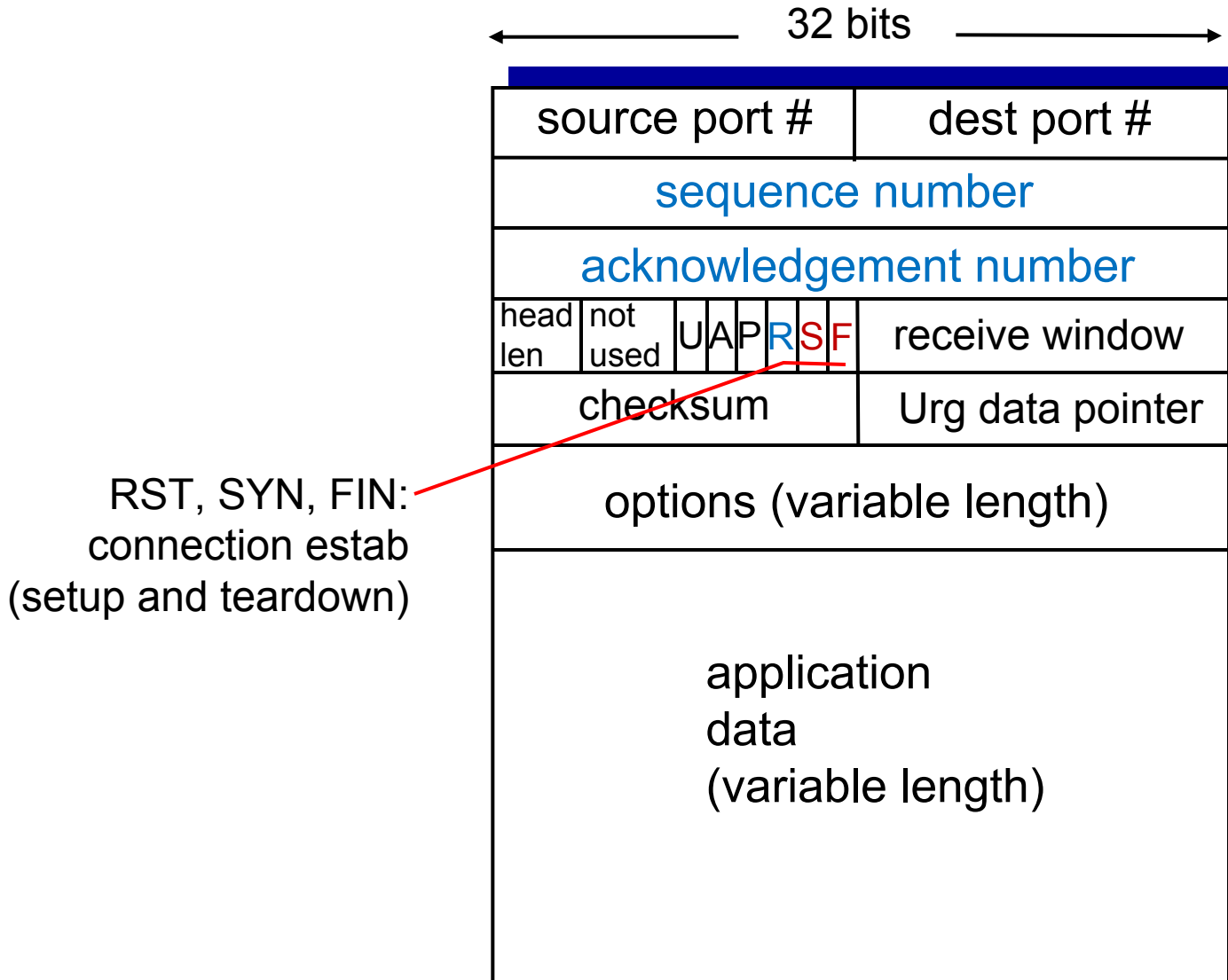
# Agreeing to establish a connection

3-way handshake:



- ❖ T1: B knows A's transmitter and B's receiver is OK
- ❖ T2: A knows A's transceiver and B's transceiver is OK, B has no more information than T1
- ❖ T3: Both A and B know their transceiver are OK, they can start the communication!

# TCP segment structure



# TCP 3-way handshake

## client state

LISTEN

SYNSENT

ESTAB

choose init seq num, x  
send TCP SYN msg



SYNbit=1, Seq=x

SYNbit=1, Seq=y  
ACKbit=1; ACKnum=x+1

received SYNACK(x)  
indicates server is live;  
send ACK for SYNACK;  
this segment may contain  
client-to-server data

SYNbit=0, Seq = x+1  
ACKbit=1, ACKnum=y+1

一旦三次握手完成，客户端和服务端可以开始数据的双向传输。在随后的数据包中，SYN标志位会被清除为0



## server state

LISTEN

SYN RCVD

ESTAB

choose init seq num, y  
send TCP SYNACK  
msg, acking SYN

received ACK(y)  
indicates client is live

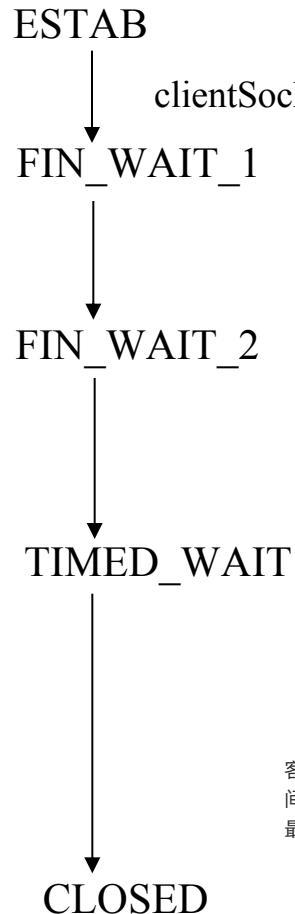
Once these three steps have been completed, the client and server hosts can send segments containing data to each other.

- In each of these **future segments, SYNbit=0**

# TCP: closing a connection

四次挥手

## client state

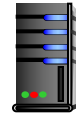


can no longer  
send but can  
receive data

wait for server  
close

timed wait

客户端在收到服务器的最终确认后，进入TIMED\_WAIT状态，表示客户端等待一段时间，确保服务器收到了ACK包。如果ACK包丢失，客户端可以重新发送确认包。最后，客户端进入CLOSED状态，完全关闭连接。



FINbit=1, seq=x

ACKbit=1; ACKnum=x+1

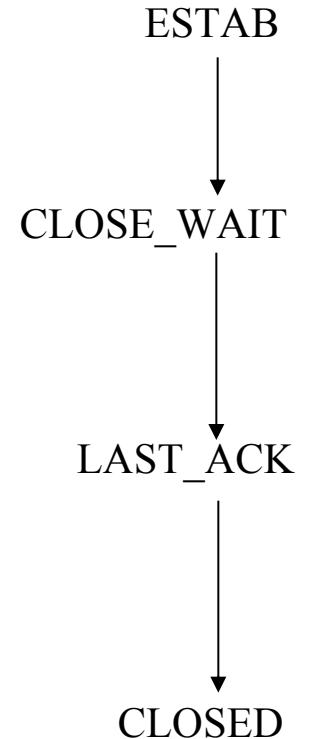
FINbit=1, seq=y

ACKbit=1; ACKnum=y+1

can still  
send data

can no longer  
send data

## server state



The TIME\_WAIT state lets the TCP client resend the final acknowledgment in case the ACK is lost.

# TCP: closing a connection

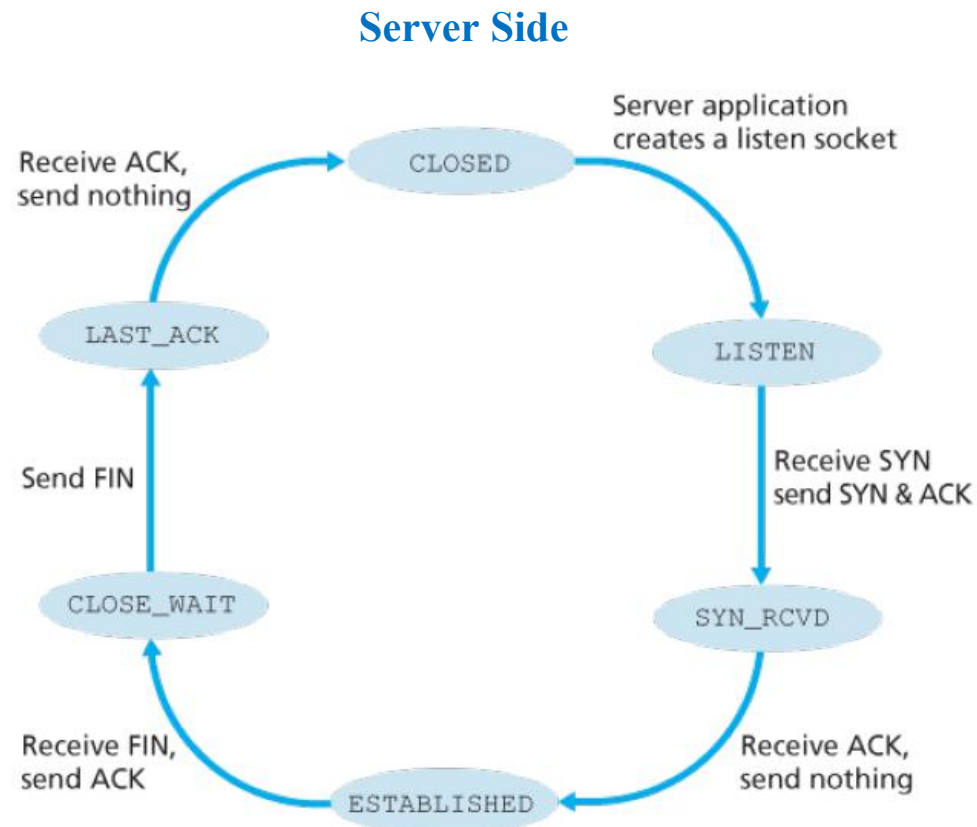
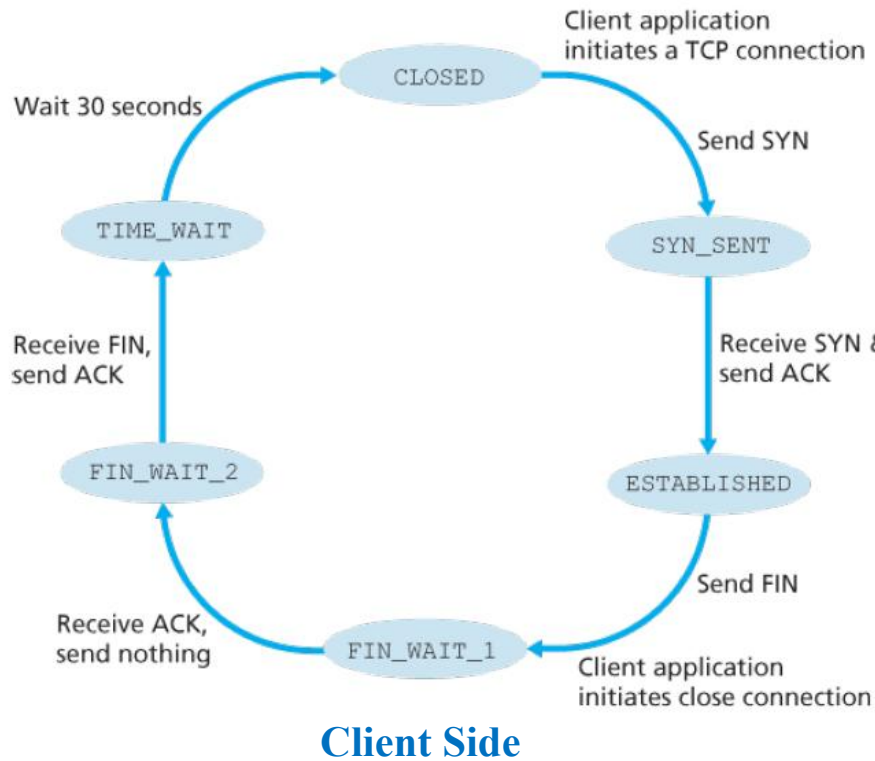
- ❖ Four-way handshaking
  - **Either** of the two processes participating in a TCP connection can end the connection.
- ❖ client, server each close their side of connection
  - send TCP segment with FIN bit = 1
- ❖ respond to received FIN with ACK

两个方向都需要关闭；所以不像建立连接的时候SYN和ACK可以合

- ❖ Why FIN and ACK **can not be sent in one msg** as SYNACK in connection establishment?
  - The other side may still have packets need to be sent. It can not send FIN until the transmission is finished.

这是因为在连接关闭时，另一方可能仍然需要发送一些数据。只有在完成所有数据传输后，才能发送FIN包来表示连接的结

# TCP States



# Reset Segment

When a host receives a TCP segment whose port numbers or source IP address do not match with any of the ongoing sockets.

- ❖ Then the host will send a special reset segment to the source.  
RST flag bit is set to 1.
- ❖ “I don’t have a socket for that segment. Please do not resend the segment.”



# Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control